# Mastering Augmented Reality with Unity

Prof. Jooho Cha, Ph.D.
jhcha@chungwoon.ac.kr

Dept. of Multimedia Science
Chungwoon University






# Chapter 6. Refreshing C# Concepts with Unity

- Introduction
  - Understanding C# concepts with Unity and the capabilities of different components will allow us to build any interaction that can be done within an AR world.
  - This chapter goes through all the essential components we discussed before and attempts to manipulate the values to build a simple top-down shooter game.
    - We will be using this game to understand a simple structure of a 3D application, moving through different scenes, using physics, coded animation, audio, and particle effects.

# Chapter 6. Refreshing C# Concepts with Unity

- Structure
  - Setting up the scene
  - Movement of the player
  - Creating a bullet for the player to shoot
  - Shooting bullets
  - Setting up the enemy
  - Setting up the player
  - Spawning and destroying enemies
  - Game UI
  - Adding audio and polishing



# Chapter 6. Refreshing C# Concepts with Unity

- Objectives
  - Understanding how to use different components for various system behaviors.
  - Understanding how to use design patterns for accessing our code.
  - Analyzing use cases of different components.
  - Understanding the polishing process of an application.
  - Making an application ready to be published as a first prototype.

# Setting up the scene

- Our requirement is to build a simple top-down shooter game.
- We will be creating a straightforward version of it but in 3D.
- We will be controlling a spaceship that shoots bullets in timely intervals and keeps shooting at the enemy spaceships that are coming at us from the opposite side.
- Occasionally, the enemy ships will also shoot at us, and we can block the bullets by shooting at them.

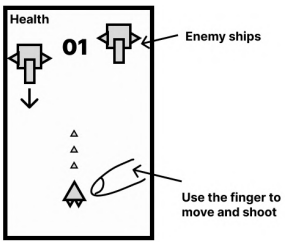



*Figure 6.1: Simple sketch of the final output of the game*

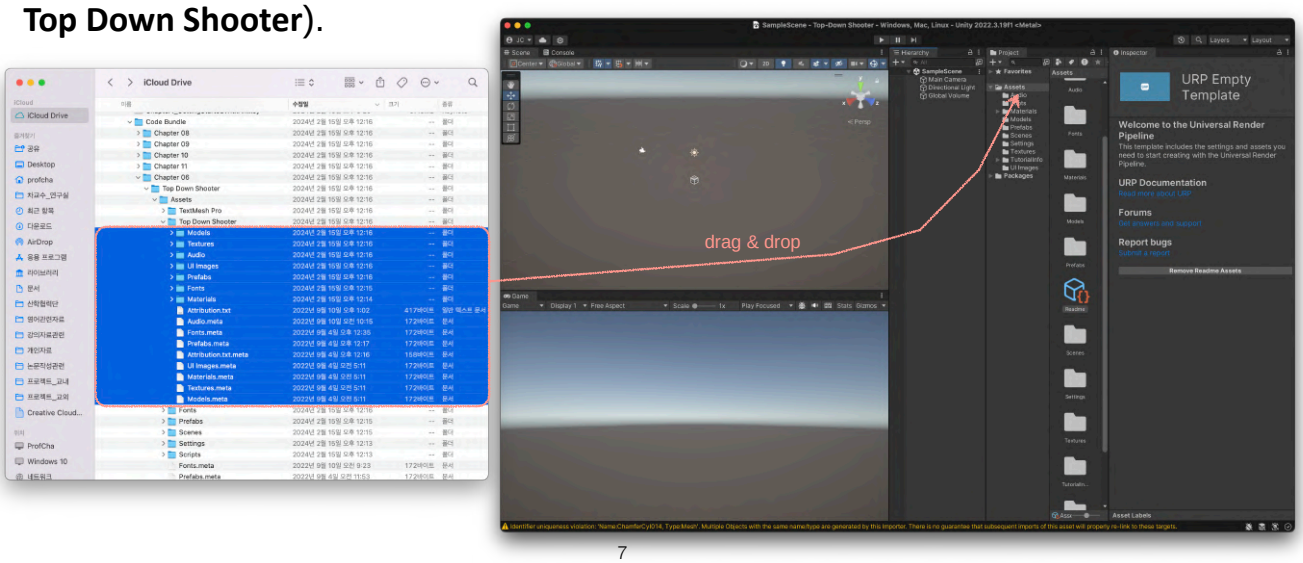


# Setting up the scene

- There will be different enemy ships with different levels of difficulty.
- The goal is to get the highest score possible during a single run.
- Let us create a new URP project in Unity and load up the sample scene.
- It is recommended to download all the assets files provided with the book before trying out this chapter.

# Setting up the scene

- Import the Top-Down Shooter folder (**Chapter 06** | **Top Down Shooter** | **Assets** | **Top Down Shooter**).






# Setting up the scene

- We need a player, enemies, bullets, and an enemy spawner to generate enemies.
- For the moment, we can use primitives to set up the basic scene.
- Create a **Sphere** for the enemy with a scale of a unit vector (1,1,1), another **Sphere** for the player, a small **Cube** for the bullet, and a big **Cube** behind the enemy.
- Rename them to **Enemy**, **Player**, **Bullet**, and **Enemy Spawner**, respectively.

# Setting up the scene

- The **Main Camera** should be moved to the top of the scene, looking down at the player as shown in Figure 6.2:
- The two cubes in the scene should be out of range of the **Main Camera**.



# Setting up the scene

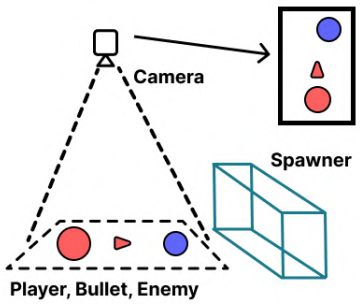
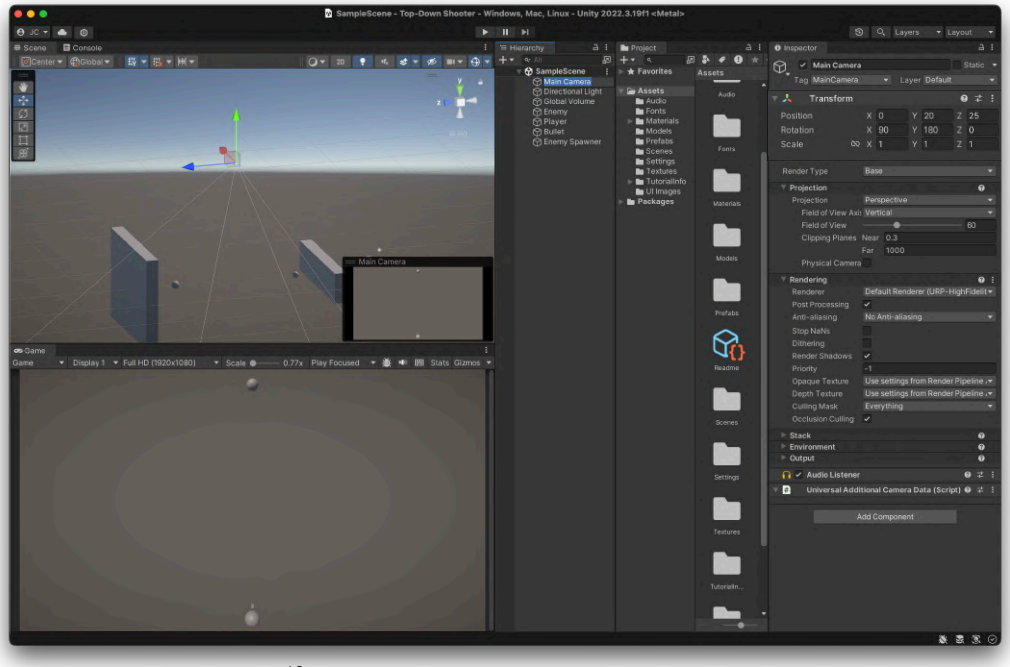



*Figure 6.2: Sketch of the Scene setup*



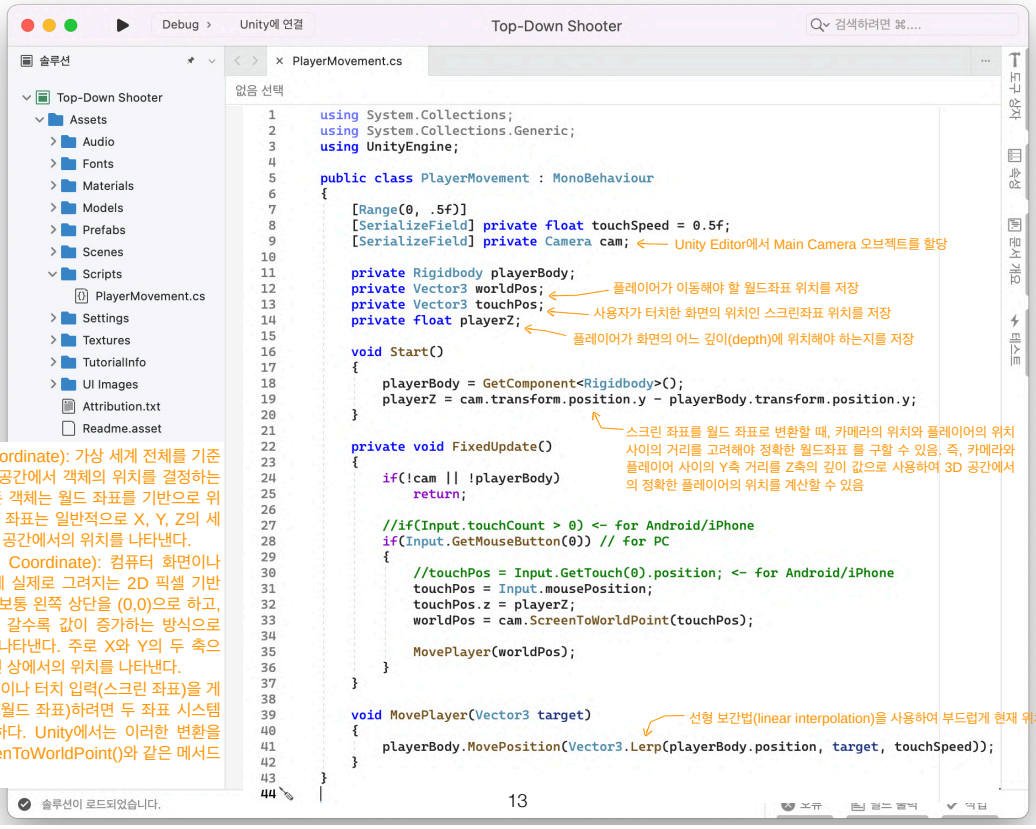
# Setting up the scene

- The player's forward direction is towards the enemy and the enemy's forward direction is towards the player.
- We will maintain the player's position in screen space and move enemies and backgrounds to visually mimic the player's endless forward motion.



# Movement of the player

- Add a **Rigidbody** to the player so that we can use physics to move the player.
- Set the **Radius** of the **Sphere Collider** to 0.5.
- And create a C# script and name it **PlayerMovement**.
  - This class is going to capture your touch input and move the player.

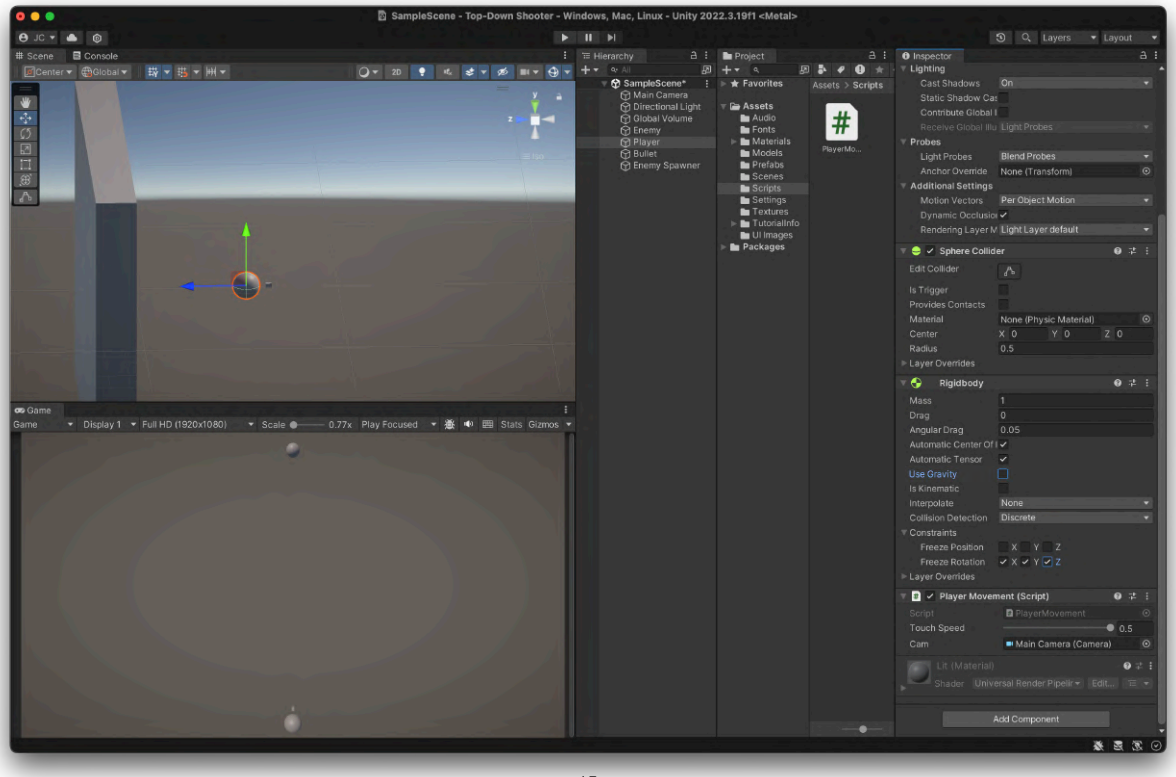



- 월드 좌표(World Coordinate): 가상 세계 전체를 기준으로 하며, 3D 가상 공간에서 객체의 위치를 결정하는데 사용된다. 즉, 모든 객체는 월드 좌표를 기반으로 위치를 갖게 된다. 월드 좌표는 일반적으로 X, Y, Z의 세 축으로 구성되며, 3D 공간에서의 위치를 나타낸다.
- 스크린 좌표(Screen Coordinate): 컴퓨터 화면이나 모바일 기기의 화면에 실제로 그려지는 2D 픽셀 기반의 좌표 시스템이다. 보통 왼쪽 상단을 (0,0)으로 하고, 오른쪽과 아래쪽으로 갈수록 값이 증가하는 방식으로 화면의 픽셀 위치를 나타낸다. 주로 X와 Y의 두 축으로 구성되며, 2D 화면 상에서의 위치를 나타낸다.
- 사용자의 마우스 클릭이나 터치 입력(스크린 좌표)을 게임 내의 객체에 적용(월드 좌표)하려면 두 좌표 시스템 사이의 변환이 필요하다. Unity에서는 이러한 변환을 위해 Camera.ScreenToWorldPoint()와 같은 메서드를 제공한다.



# Movement of the player

- Now select the **Player** game object and attach the **PlayerMovement** script and attach the **Main Camera** to the **Cam** field.
- In the **Rigidbody** component, disable the **Use Gravity** and enable rotation constraints for all **X**, **Y**, and **Z** under the **Constraints** section.
  - This prevents the player from being pulled down by gravity.
  - This restricts the player from rotating due to external forces.






# Update() vs. FixedUpdate()

- **Update()** method:
  - The **Update()** is called once per frame. This means the call frequency varies with the game's frame rate. For instance, if the game runs at 60 frames per second, **Update()** will be called 60 times a second.
  - It is primarily used for detecting inputs, updating animations, and checking rapidly changing game states that can vary from frame to frame.
  - Since the call frequency depends on the frame rate, when dealing with time-dependent logic, it is necessary to adjust it using **Time.deltaTime**.

# Update() vs. FixedUpdate()

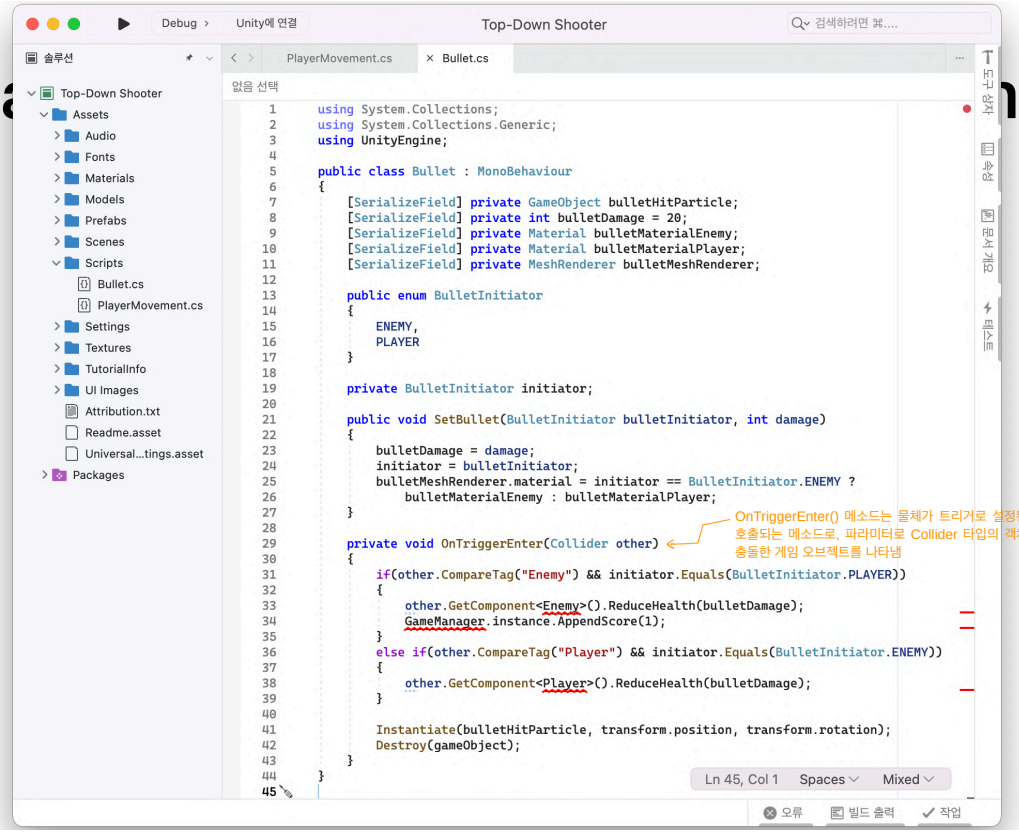
- **FixedUpdate()** method:
  - The **FixedUpdate()** function is called at fixed time intervals. It is independent of the game's frame rate and synchronizes with the physics engine's update cycle.
  - It is mainly used for calculations, movements, and physical interactions handled by the physics engine. For example, when moving an object using a **Rigidbody** component, it should be done within **FixedUpdate()**.
  - The calling interval of **FixedUpdate()** can be adjusted in the **Fixed Timestep** value (**Edit** | **Project Settings** | **Time**). This is important for maintaining consistent physics simulations in the game.



# Creating a bullet for the player to shoot

- Select the cube (named **Bullet**) object and attach a **Rigidbody** component, and disable the **Use Gravity**.
- The **Bullet** need to be instantiated by both the player and the enemy.
  - The **Bullet** should provide the damage and information about who fired the bullet (the player or the enemy).
- Create a new C# script and name it **Bullet**.

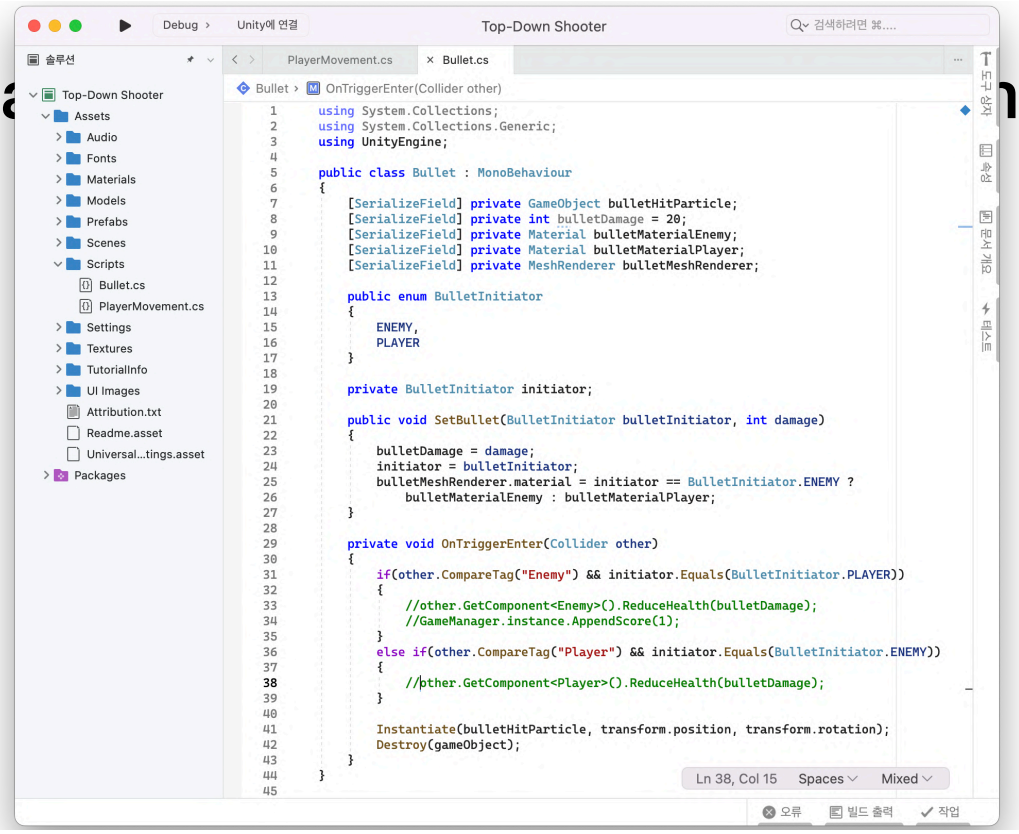
Crea hoot




# Creating a bullet for the player to shoot

- Now Let us try attaching the **Bullet** script to the **Bullet** object.
  - You should probably see errors since we reference scripts that are not available yet.
  - For the moment, you can comment on the lines that contains calls to components (**Enemy**, **GameManager**, and **Player**) that are not created.

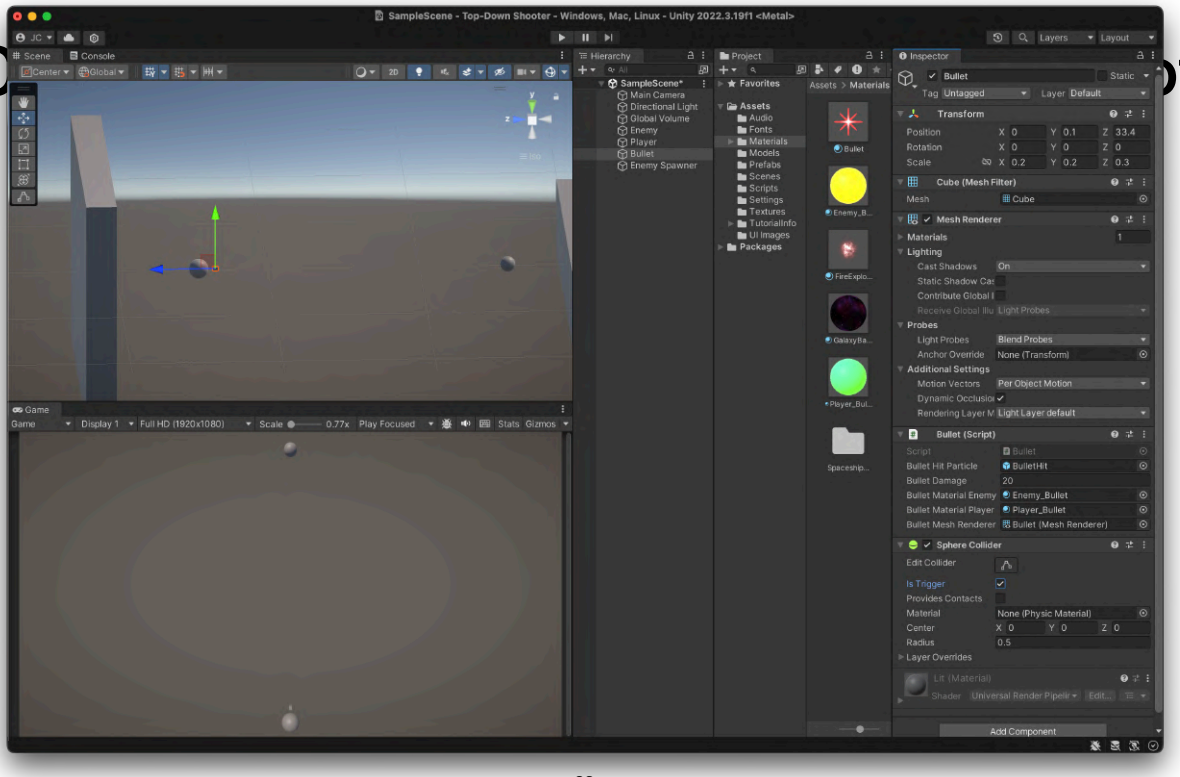
Crea hoot




# Creating a bullet for the player to shoot

- In the Inspector window of the **Bullet** object, assign a bullet hit particle (**Assets** | **Prefabs** | **BulletHit**), two materials for the player, and the enemy bullets (**Assets** | **Materials** | **Enemy_Bullet/ Player_Bullet**) from the Project window.
- Assign the **Mesh Renderer** component of the **Bullet** as the **Bullet Mesh Renderer**.
- Remove the existing **Box Collider** component and add a **Sphere Collider**. Additionally, tick **IS Trigger**.

C ot






# Shooting bullets

- Let us see how to instantiate bullets and shoot at the enemy:

1. Before creating a new component for this, we need to convert the **Bullet** into a prefab. Once done, you can remove the **Bullet** from the screen.

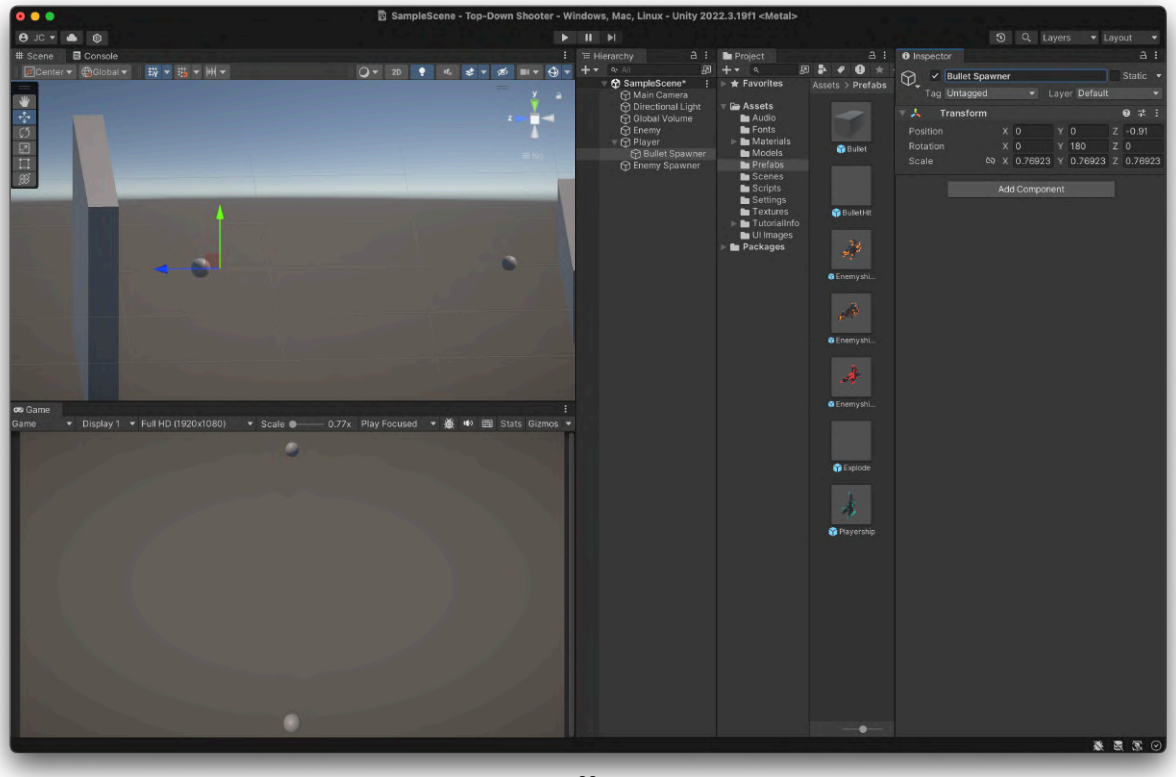
2. Now create an empty game object (**Bullet Spawner**) within the **Player**, and move it in front of the **Player**. This is going to be a reference point for the bullet to be instantiated. Because the object is inside the **Player** as a child, it moves with the **Player**.

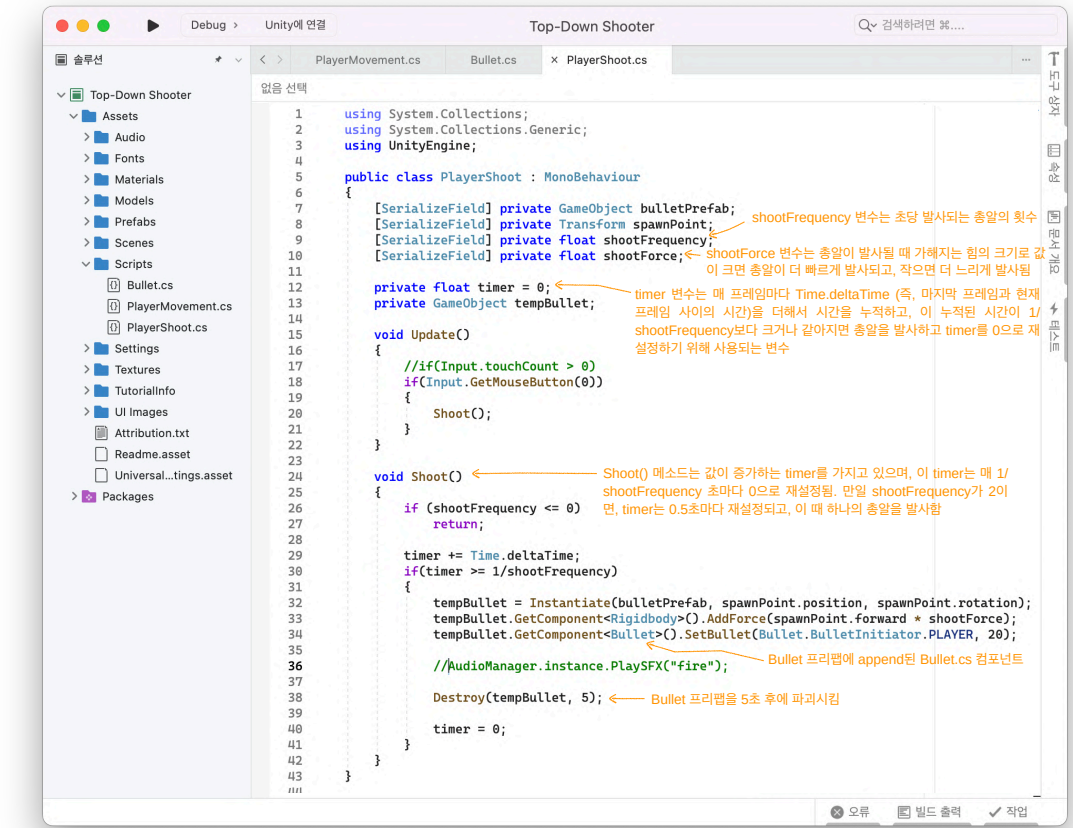
# Shooting bullets

3. Rotate the **Bullet Spawner** object so that its local forward direction is pointing forward. If the **Rotation** value of the **Player** is **0** for **Y**, you can set he **Rotation** value of the **Bullet Spawner** to **0** for **Y** as well. This will cause the **Bullet** to fire in the direction the **Player** is looking.

4. Create a new C# script and name it **PlayerShoot**. This component will instantiate and shoot bullets with a given frequency while the screen is being touched.



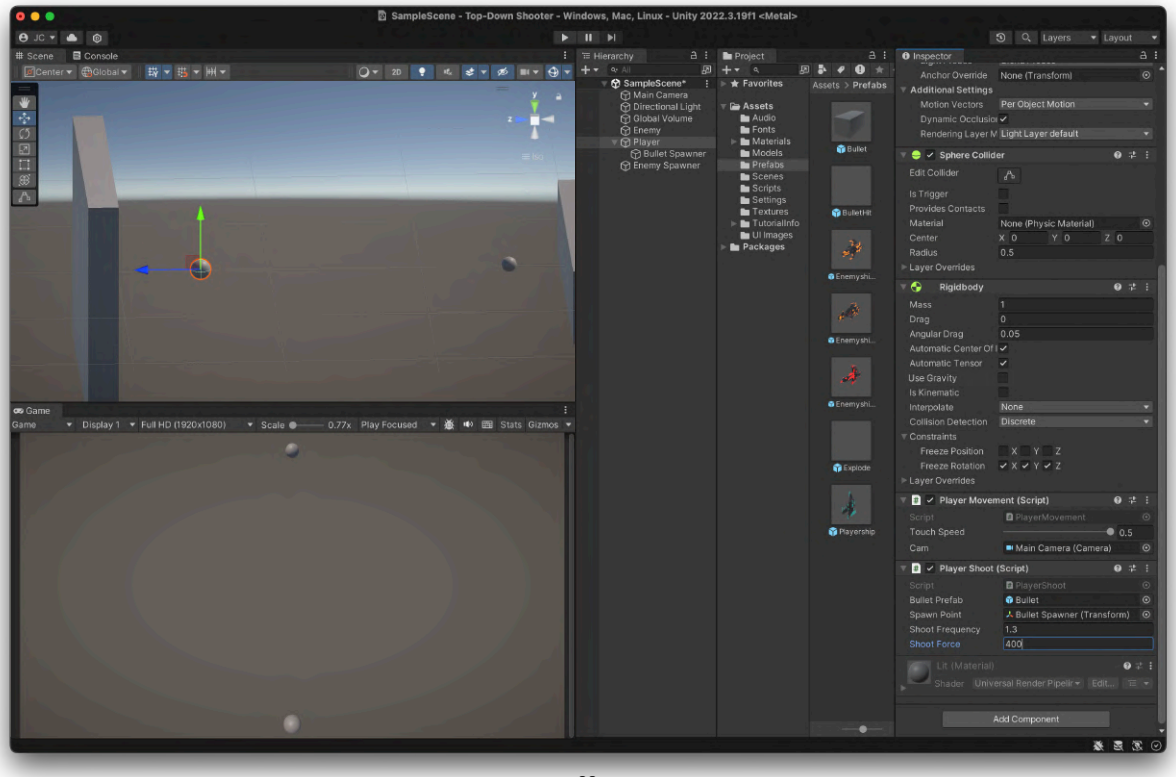





# Shooting bullets

5. Before resetting the timer to zero, we register the bullet to be destroyed after 5 second. If we do not destroy it and if the bullet does not hit an enemy, it will be loaded into memory, which will increase memory usage and hinder the performance.

6. Go back to Unity Editor and attach the **PlayerShoot** script to the **Player**. And assign the **Bullet** prefab to the **Bullet Prefab** option and the **Bullet Spawner** to the **Spawn Point** option. And set the **Shoot Frequency** to **1.3** (13 bullets per 10 seconds) and the **Shoot Force** to **400**.






# Setting up the enemy

- Once the enemy is spawned, it will move in the forward direction with a given speed and shoot a bullet in random time frequencies.
- The enemy will have a health value that will be reduced when it collides with a bullet.
  - If the health goes below 0, the enemy will explode and destroy.
  - In order to indicate the enemy's health, we are going to use a simple world space canvas with a tiny health bar attached to the top of the enemy.

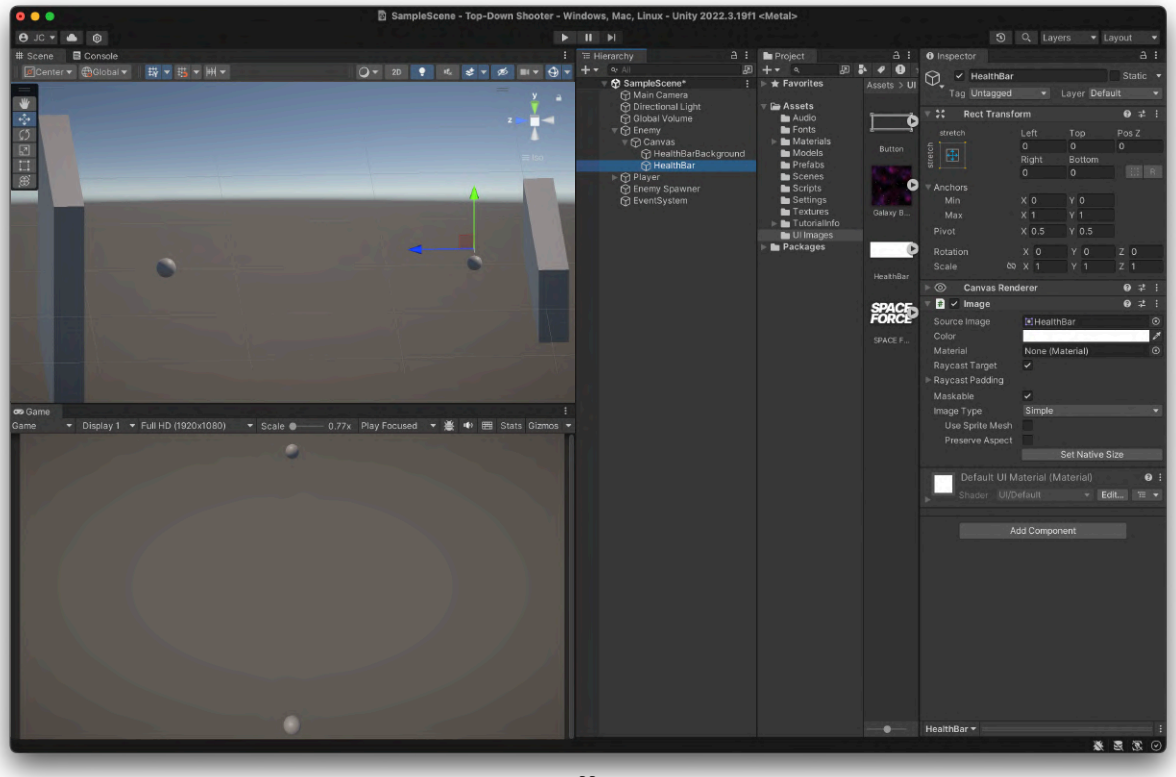
# Setting up the enemy

1. Select the **Enemy** game object. Right-click and Add **UI** | **Canvas**.
2. Select the **Canvas** object. Change the **Render Mode** to **World Space** in the **Canvas** component and set all **Scale** to **0.01** in the **Rect Transform** component.
3. We will be adding two **Image** game objects in the **Canvas** by using **UI** | **Image**. Name them to **HealthBarBackground** and **HealthBar**. For both images, set **0** for **Min** and **1** for **Max**, while keeping the **Pivot** to **0.5** and all the **Left**, **Right**, **Top**, and **Bottom** to **0** in the **Rect Transform** component.

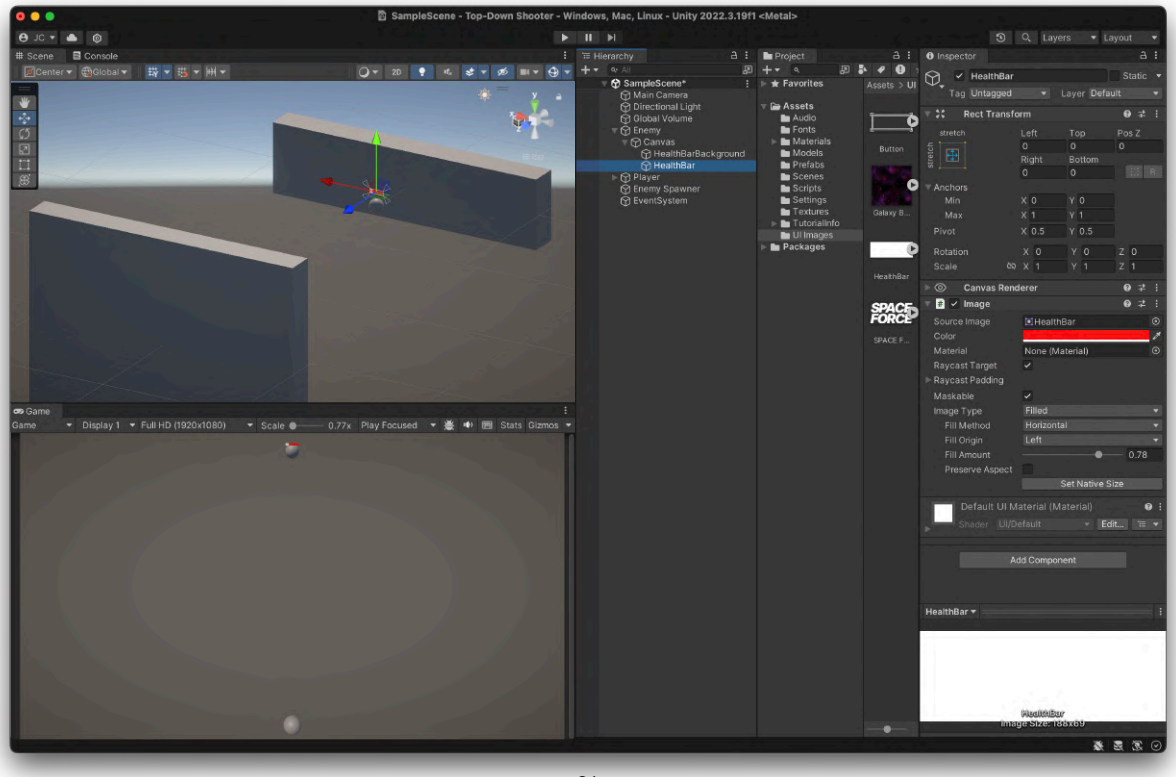


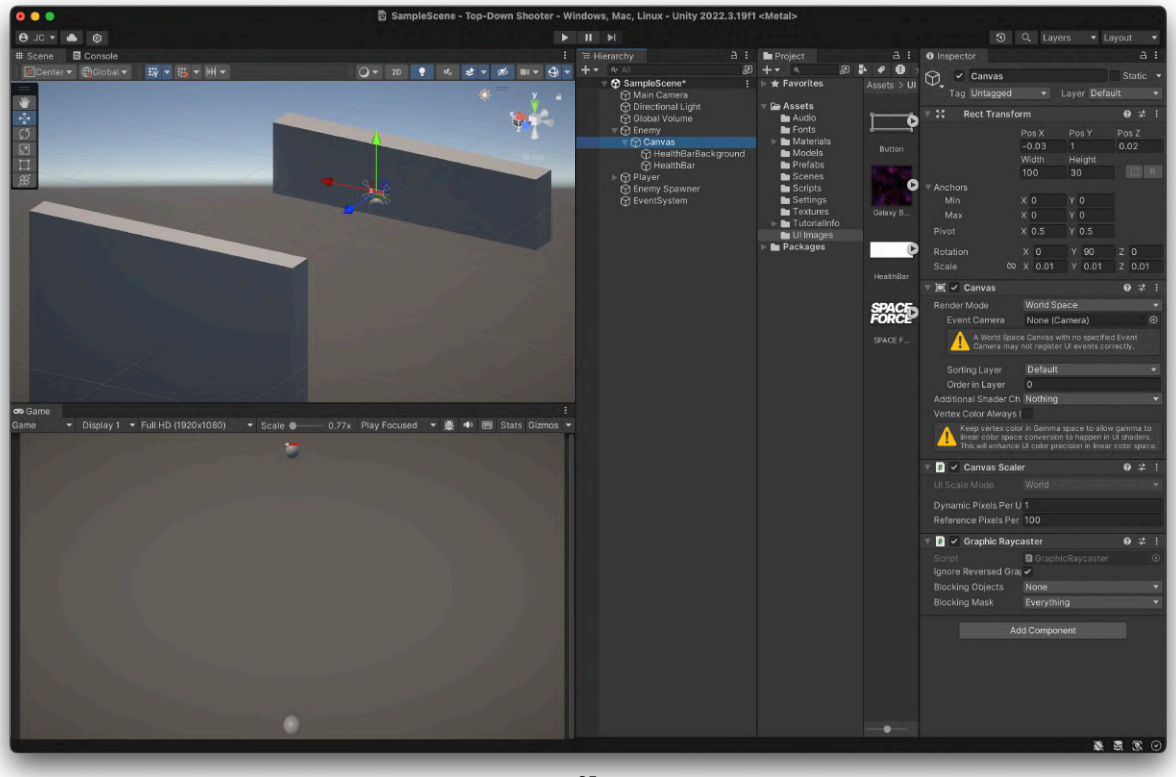
# Setting up the enemy

4. Now select the **HealthBar** and make sure that it is at the bottom of the **Canvas**. Change the **Source Image** to **HealthBar** sprite from the **Assets** (**UI Images** | **HealthBar**) in the **Image** component.
5. Set the **Color** to red and the **Image Type** to **Filled**. Change the **Fill Method** to **Horizontal**. Now if we change the **Fill Amount**, we will be able to see a small progress bar that would look like a mini health bar.
6. Position the **Canvas** above the **Enemy** by adjusting the **Y** value so that the **Canvas** is visible from the top camera.


SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
Hierarchy
Project
Inspector
HealthBar
Rect Transform
Canvas Renderer
Image
Source Image
Color
Material
Raycast Target
Raycast Padding
Maskable
Image Type
Use Sprite Mesh
Preserve Aspect
Set Native Size
Default UI Material (Material)
Add Component



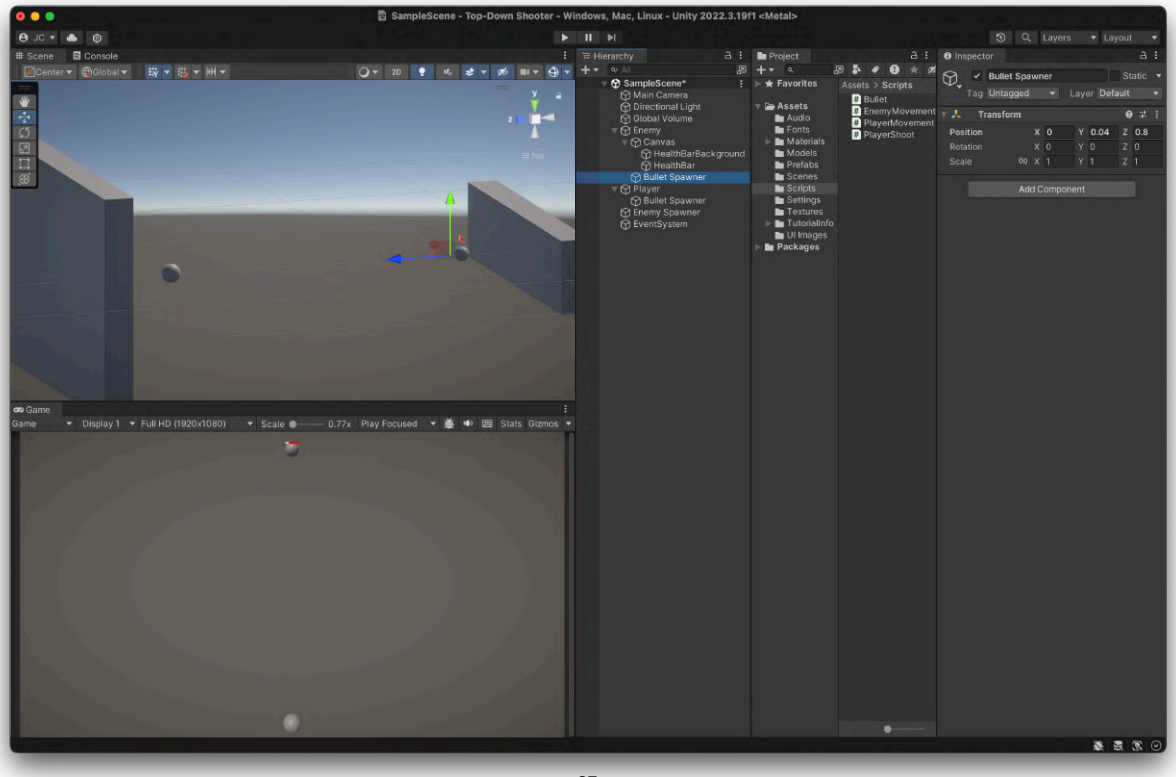

SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
Hierarchy
Project
Inspector
HealthBar
Rect Transform
Canvas Renderer
Image
Source Image
Color
Material
Raycast Target
Raycast Padding
Maskable
Image Type
Fill Method
Fill Origin
Fill Amount
Preserve Aspect
Set Native Size
Default UI Material (Material)
Add Component



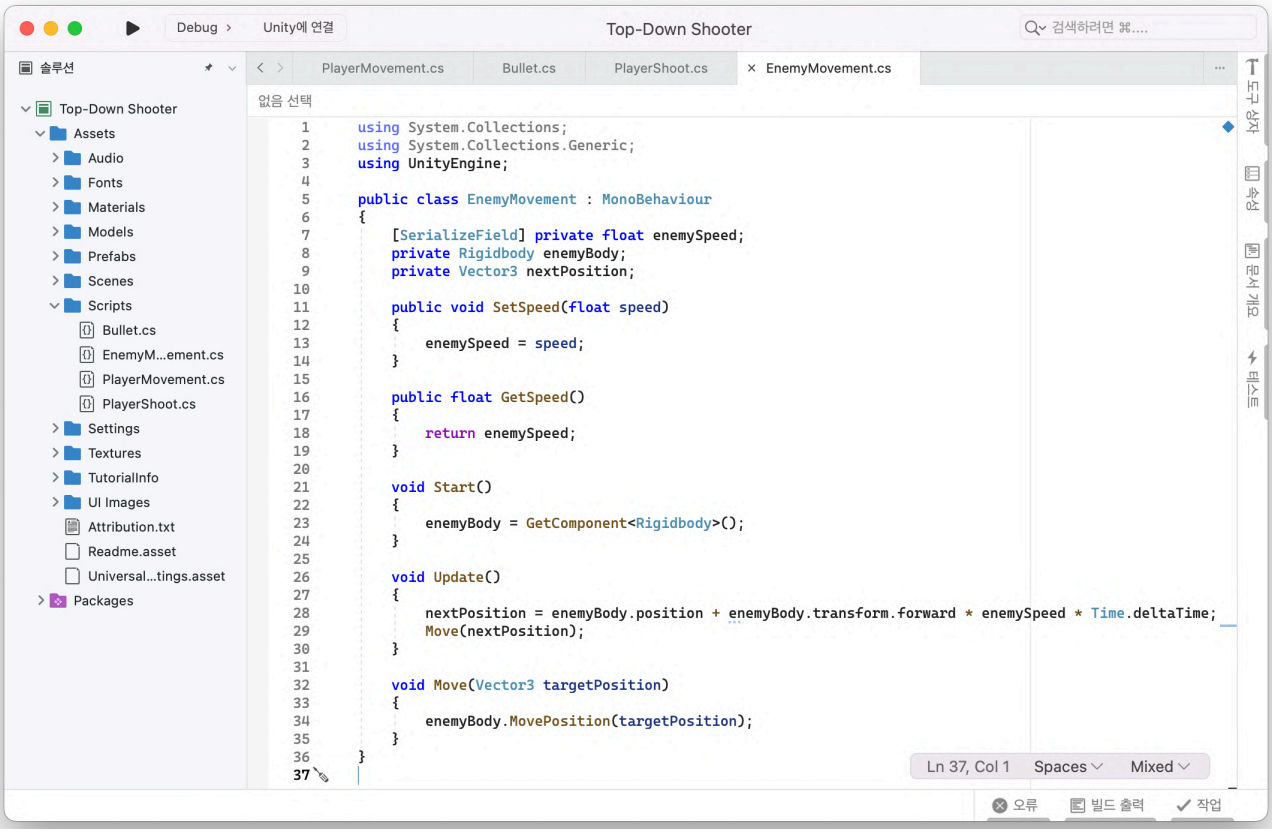



# Setting up the enemy

7. Now like the **Player**'s spawn point, add an empty game object (**Bullet Spawner**) inside the **Enemy** as well, and make sure that it is rotated towards the forward direction of the **Enemy**. The **Enemy** can look in the opposite direction as the **Player**. Therefore, set the **Rotation** value of the **Bullet Spawner** to **180** for **Y** as well. This will cause the **Bullet** to fire in the direction of the **Player**.
8. Create a new C# script and name it **EnemyMovement**.
9. In the script, we also add a couple of **Set** and **Get** methods to manage the **speed** variable. The rigid body added to the enemy will be cached at the **Start** method using the **GetComponent** method.


SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
Hierarchy
Project
Inspector
Bullet Spawner
Transform
Add Component
Game

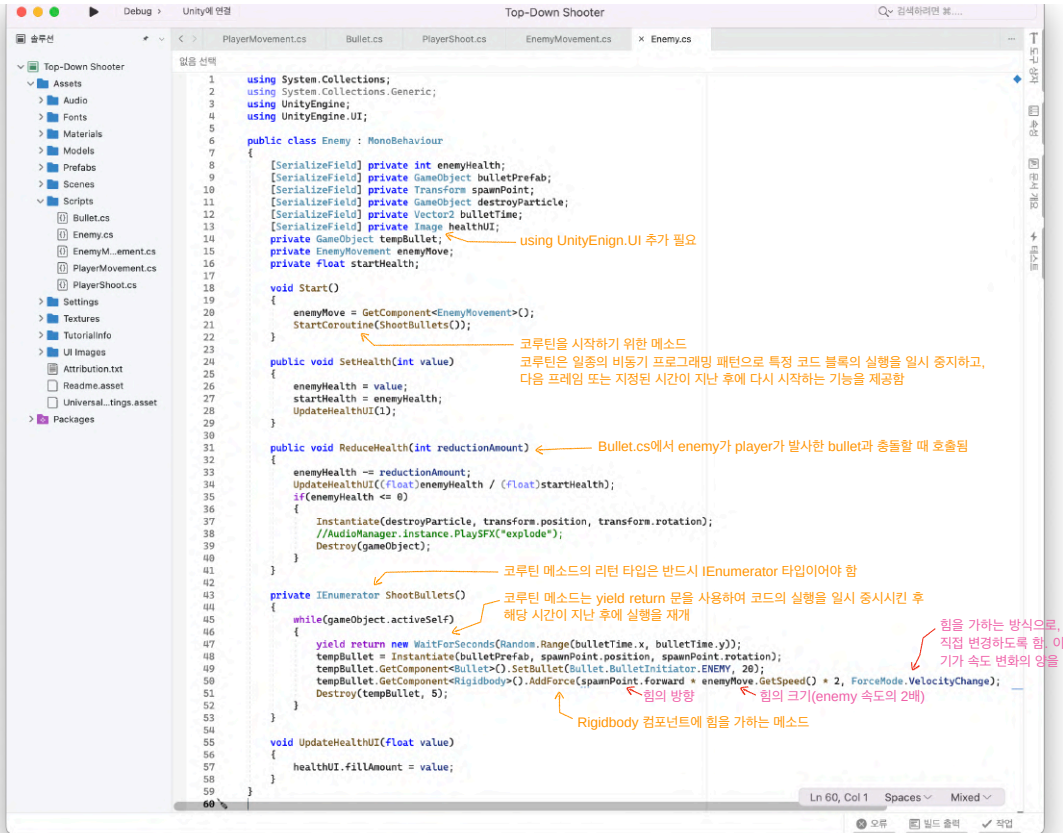



Debug
Unity에 연결
Top-Down Shooter
검색하려면 ⌘....
솔루션
Top-Down Shooter
Assets
Audio
Fonts
Materials
Models
Prefabs
Scenes
Scripts
Bullet.cs
EnemyM...ement.cs
PlayerMovement.cs
PlayerShoot.cs
Settings
Textures
TutorialInfo
UI Images
Attribution.txt
Readme.asset
Universal...tings.asset
Packages
PlayerMovement.cs
Bullet.cs
PlayerShoot.cs
EnemyMovement.cs
없음 선택
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class EnemyMovement : MonoBehaviour
6 {
7 [SerializeField] private float enemySpeed;
8 private Rigidbody enemyBody;
9 private Vector3 nextPosition;
10
11 public void SetSpeed(float speed)
12 {
13 enemySpeed = speed;
14 }
15
16 public float GetSpeed()
17 {
18 return enemySpeed;
19 }
20
21 void Start()
22 {
23 enemyBody = GetComponent<Rigidbody>();
24 }
25
26 void Update()
27 {
28 nextPosition = enemyBody.position + enemyBody.transform.forward * enemySpeed * Time.deltaTime;
29 Move(nextPosition);
30 }
31
32 void Move(Vector3 targetPosition)
33 {
34 enemyBody.MovePosition(targetPosition);
35 }
36 }
37
Ln 37, Col 1
Spaces
Mixed
오류
빌드 출력
작업
도구 상자
속성
문서 개요
테스트

# Setting up the enemy

10. In order to manage the rest of the enemy functions, create another C# script and name it **Enemy**.

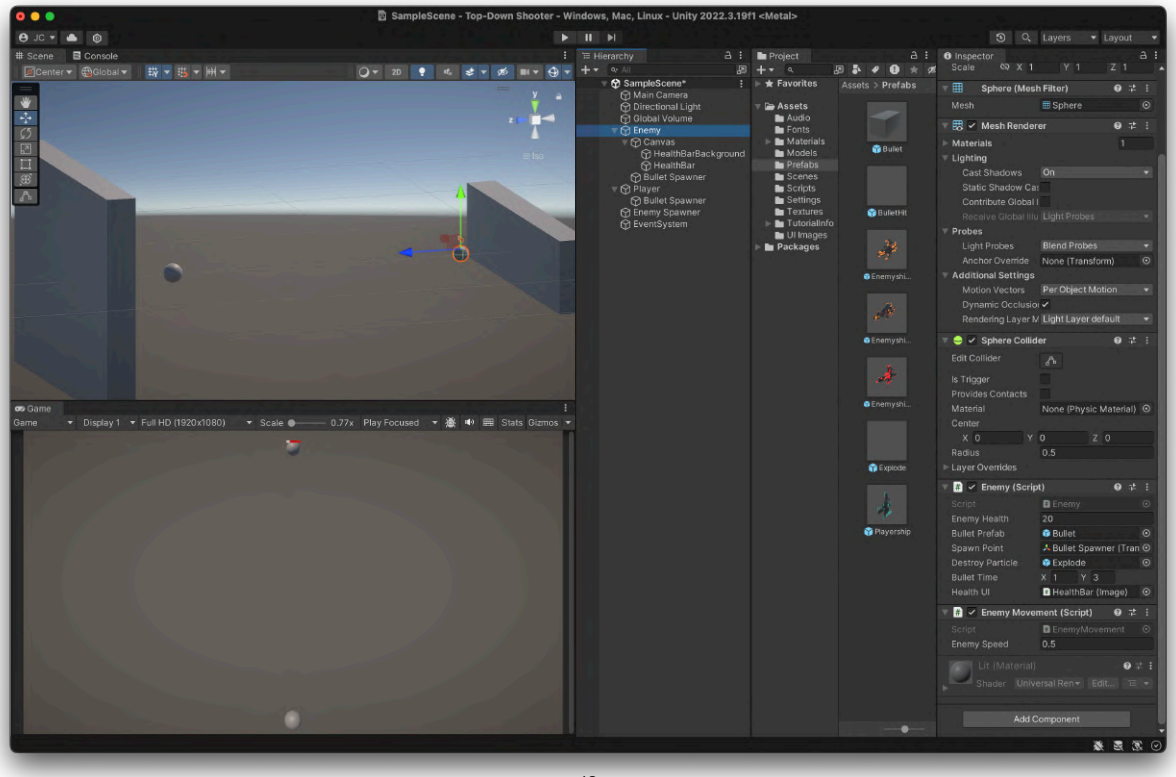




# Setting up the enemy

11. Now select the **Enemy** game object and attach both **EnemyMovement** and **Enemy** scripts. Assign all the necessary references to the **Enemy** and **EnemyMovement** components.

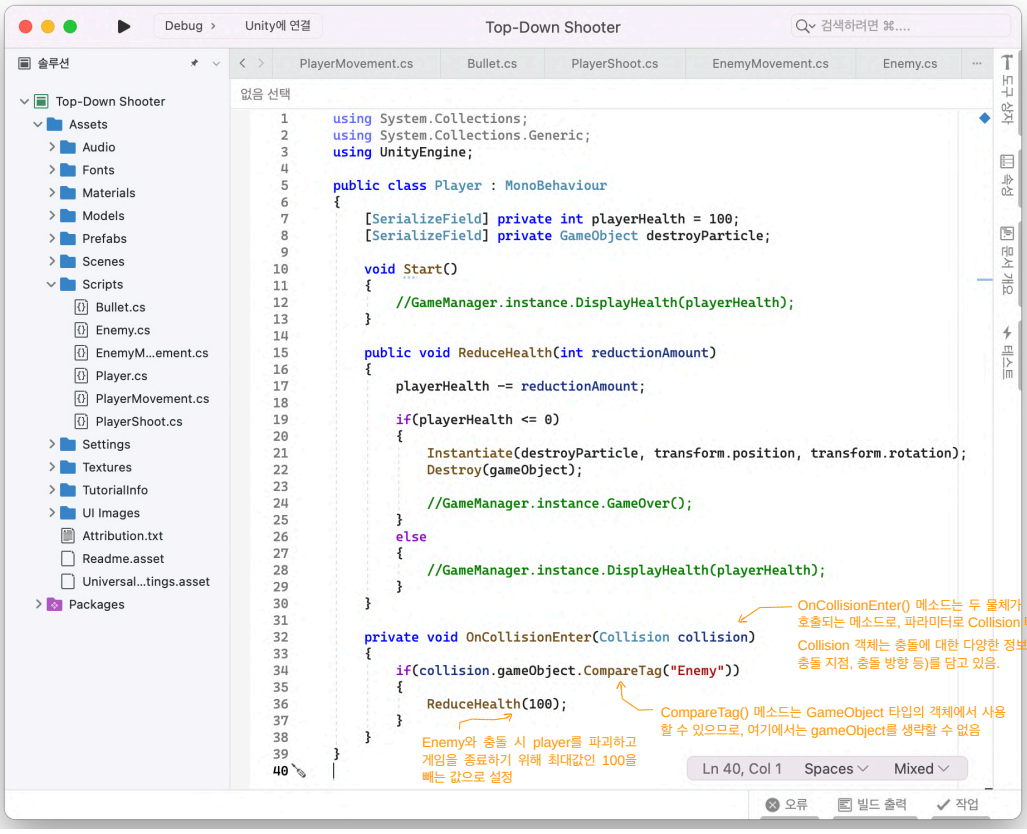




# Setting up the player

- Now, let us finalize the **Player** object by adding a new **Player** script.
  - Similar to the **Enemy** script, this will take care of the health of the player and end the game once the player gets destroyed.
  - And the player will immediately destroy if it collides with an enemy.

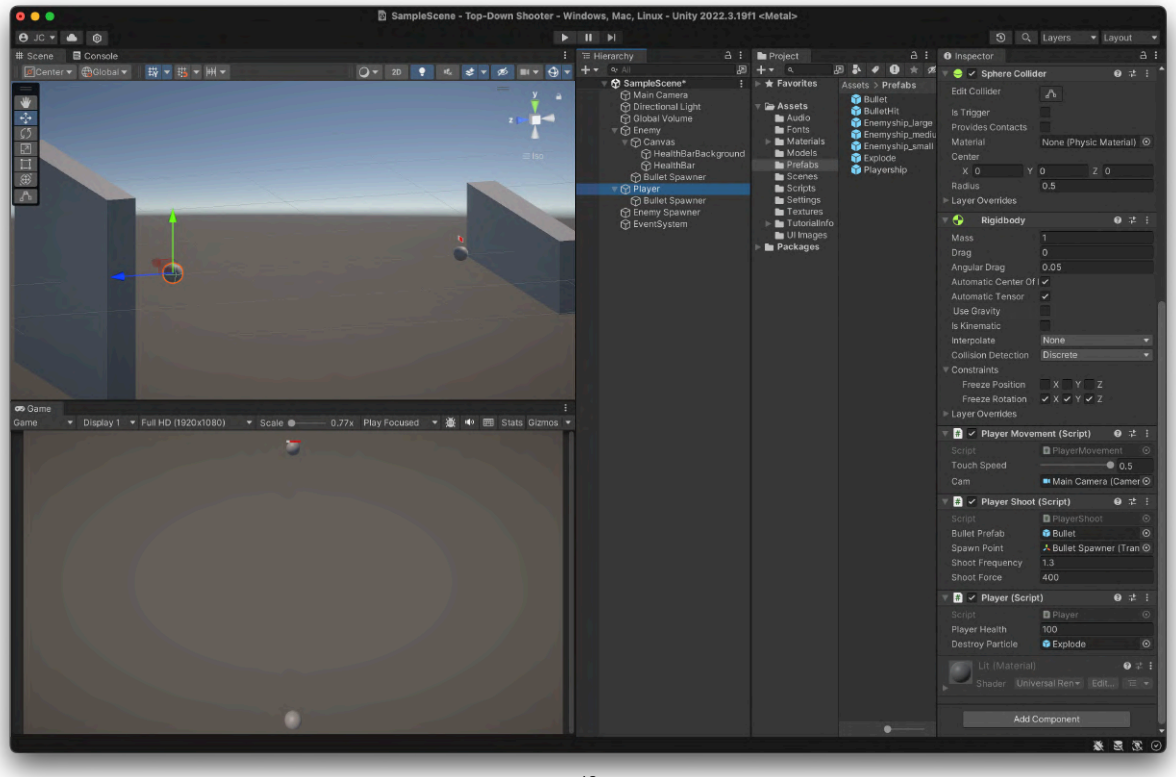


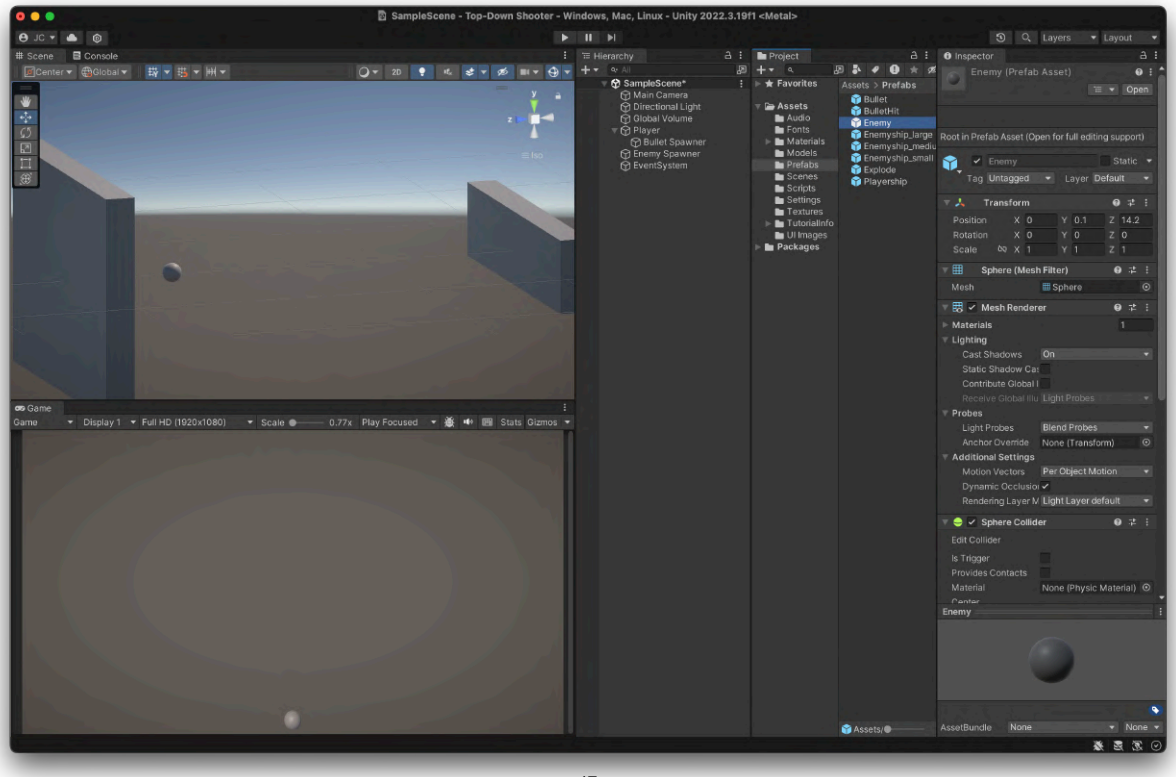


# Setting up the player

- Attach the **Player** script to the **Player** object and assign the **Destroy Particle** as **Explode**.
- Since all the base components are set up, we can convert the **Enemy** object into a prefab and remove it from the scene.





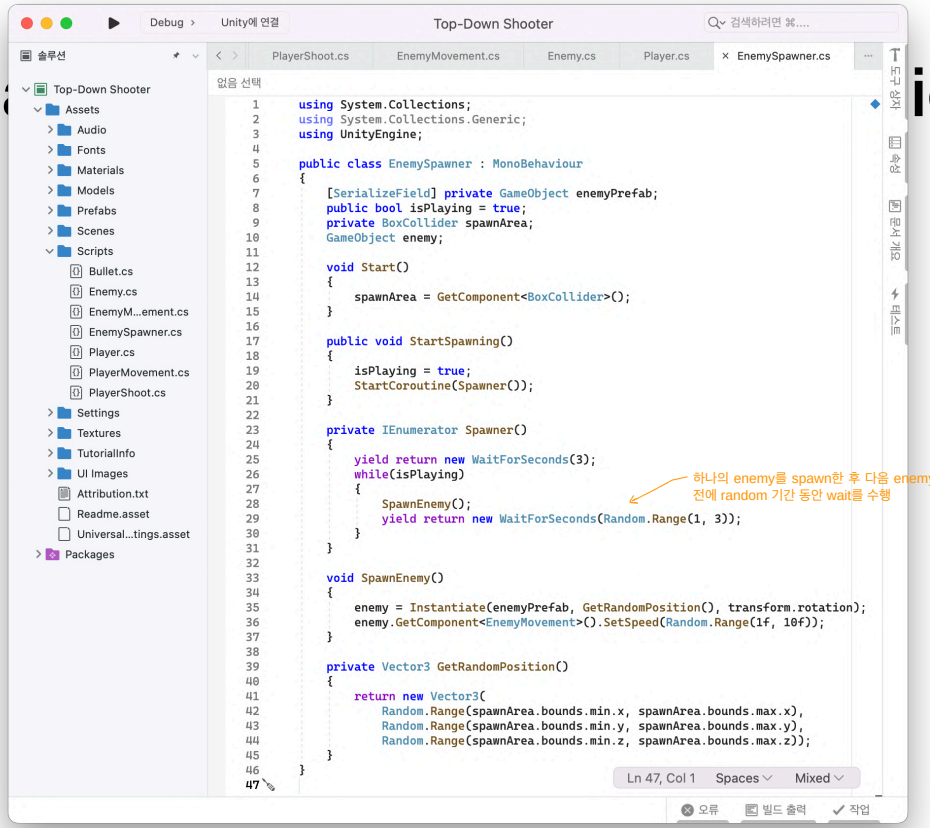


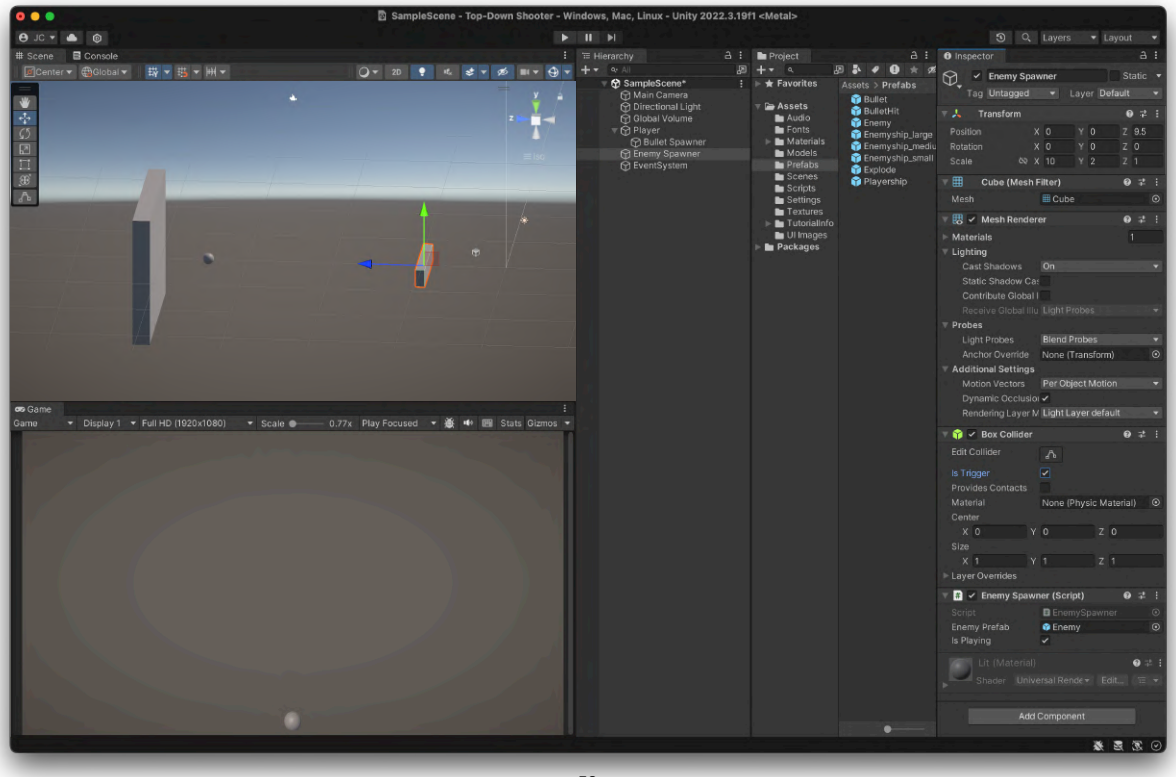



# Spawning and destroying enemies

- We will be spawning and destroying the enemies.
- Create a new C# script and name it **EnemySpawner**.
- And attach the **EnemySpawner** script to the **Enemy Spawner** game object. Make sure that the **Box Collider** of the **Enemy Spawner** object is a **Trigger** (**Is Trigger** | **Ticked**).

# Spa ies

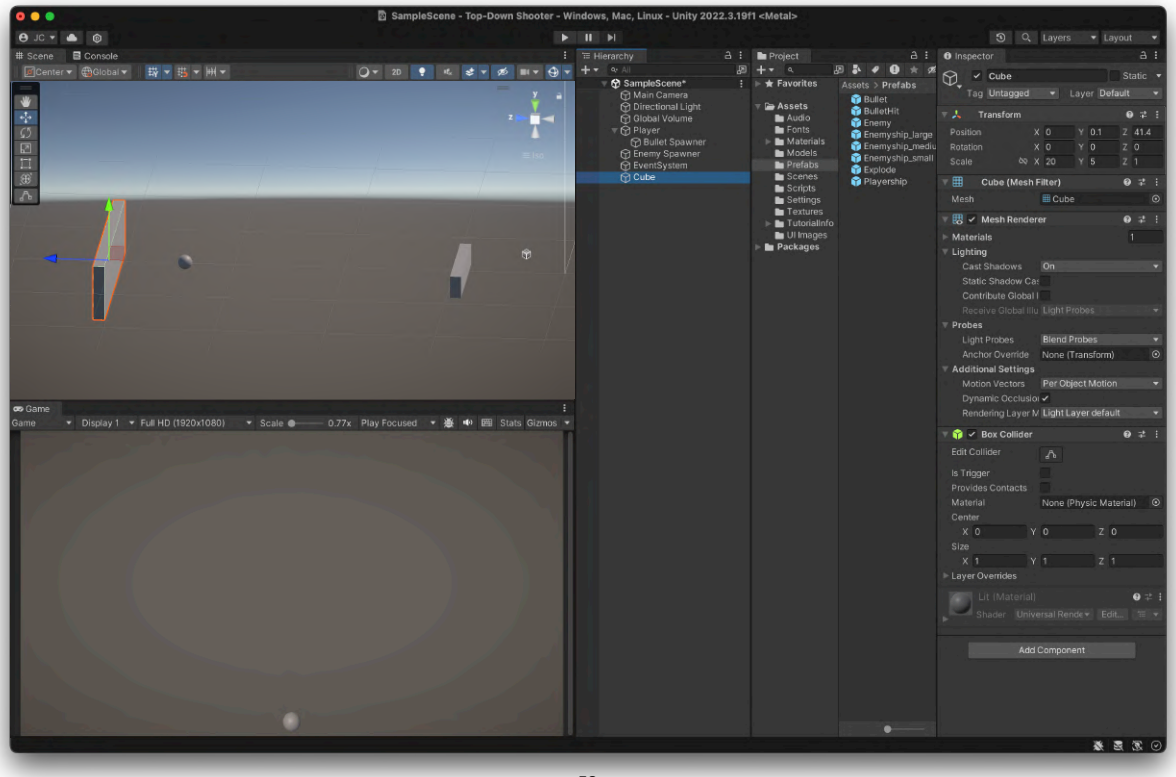

Top-Down Shooter
PlayerShoot.cs
EnemyMovement.cs
Enemy.cs
Player.cs
EnemySpawner.cs
Assets
Audio
Fonts
Materials
Models
Prefabs
Scenes
Scripts
Bullet.cs
Enemy.cs
EnemySpawner.cs
Player.cs
PlayerMovement.cs
PlayerShoot.cs
Settings
Textures
TutorialInfo
UI Images
Attribution.txt
Readme.asset
Packages
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class EnemySpawner : MonoBehaviour
{
[SerializeField] private GameObject enemyPrefab;
public bool isPlaying = true;
private BoxCollider spawnArea;
GameObject enemy;
void Start()
{
spawnArea = GetComponent<BoxCollider>();
}
public void StartSpawning()
{
isPlaying = true;
StartCoroutine(Spawner());
}
private IEnumerator Spawner()
{
yield return new WaitForSeconds(3);
while(isPlaying)
{
SpawnEnemy();
yield return new WaitForSeconds(Random.Range(1, 3));
}
}
void SpawnEnemy()
{
enemy = Instantiate(enemyPrefab, GetRandomPosition(), transform.rotation);
enemy.GetComponent<EnemyMovement>().SetSpeed(Random.Range(1f, 10f));
}
private Vector3 GetRandomPosition()
{
return new Vector3(
Random.Range(spawnArea.bounds.min.x, spawnArea.bounds.max.x),
Random.Range(spawnArea.bounds.min.y, spawnArea.bounds.max.y),
Random.Range(spawnArea.bounds.min.z, spawnArea.bounds.max.z));
}
}
하나의 enemy를 spawn한 후 다음 enemy를 spawn하기 전에 random 기간 동안 wait를 수행

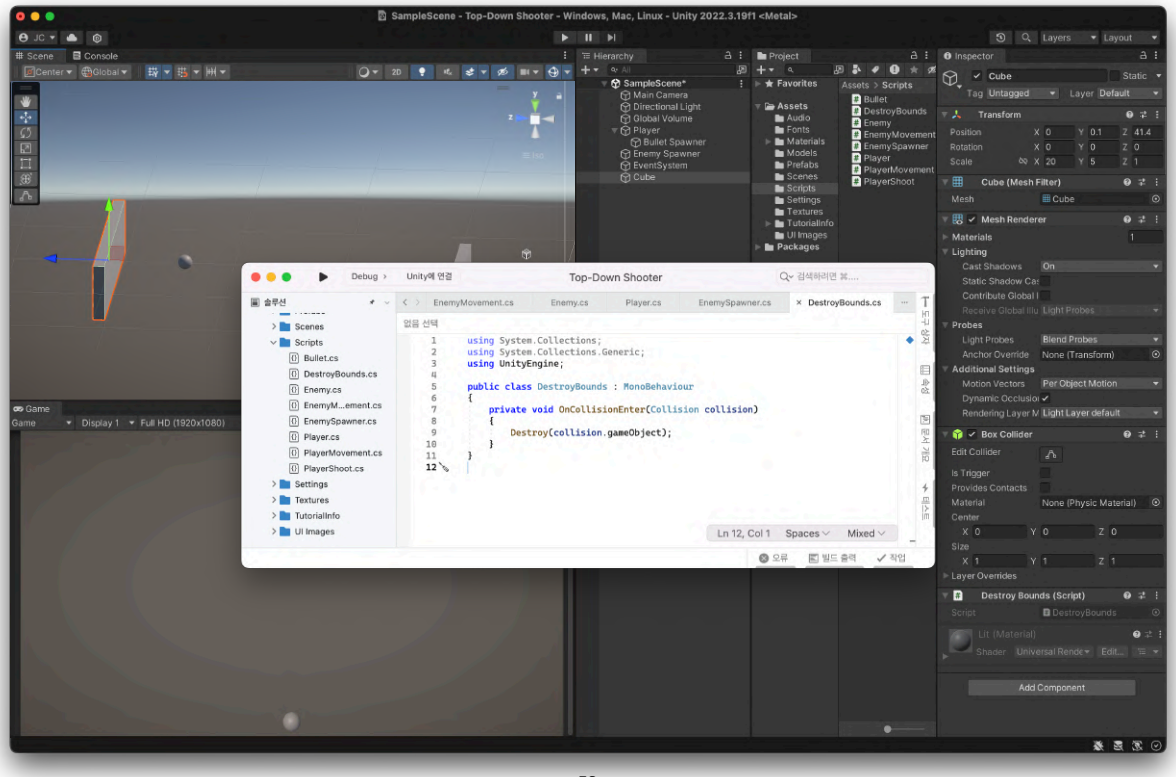

SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
Hierarchy
Project
Inspector
Enemy Spawner
Transform
Cube (Mesh Filter)
Mesh Renderer
Box Collider
Enemy Spawner (Script)
Add Component

# Spawning and destroying enemies

- Now that we can spawn enemies, there is high chance that the enemies can bypass the player and be in scene outside of the camera's range.
- Therefore, we need to destroy the enemies that go beyond the player.
  - Just like the spawner, let us create another **Cube** that is placed behind the player.
  - Create a new C# script and name it as **DestroyBounds**. Add this component to the **Cube** that we created.







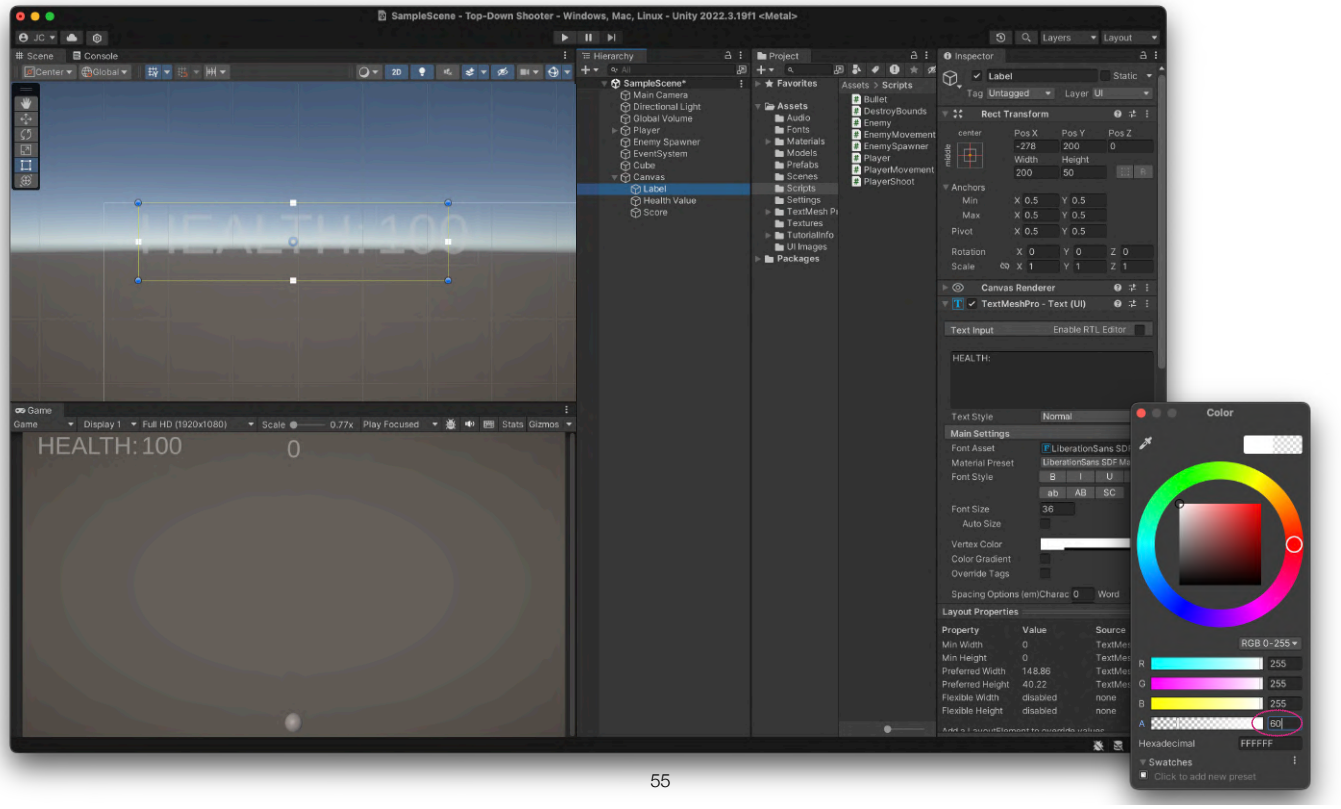



# Game UI

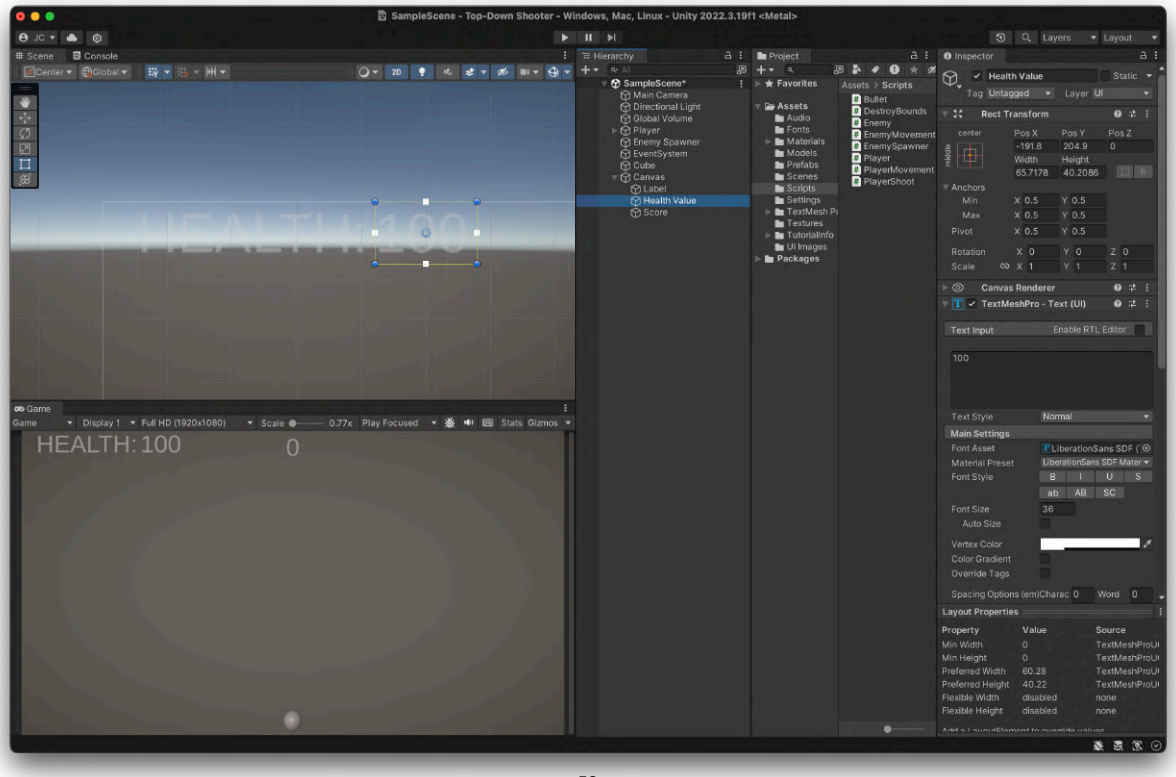
- Let us set up additional game UI objects in the scene.

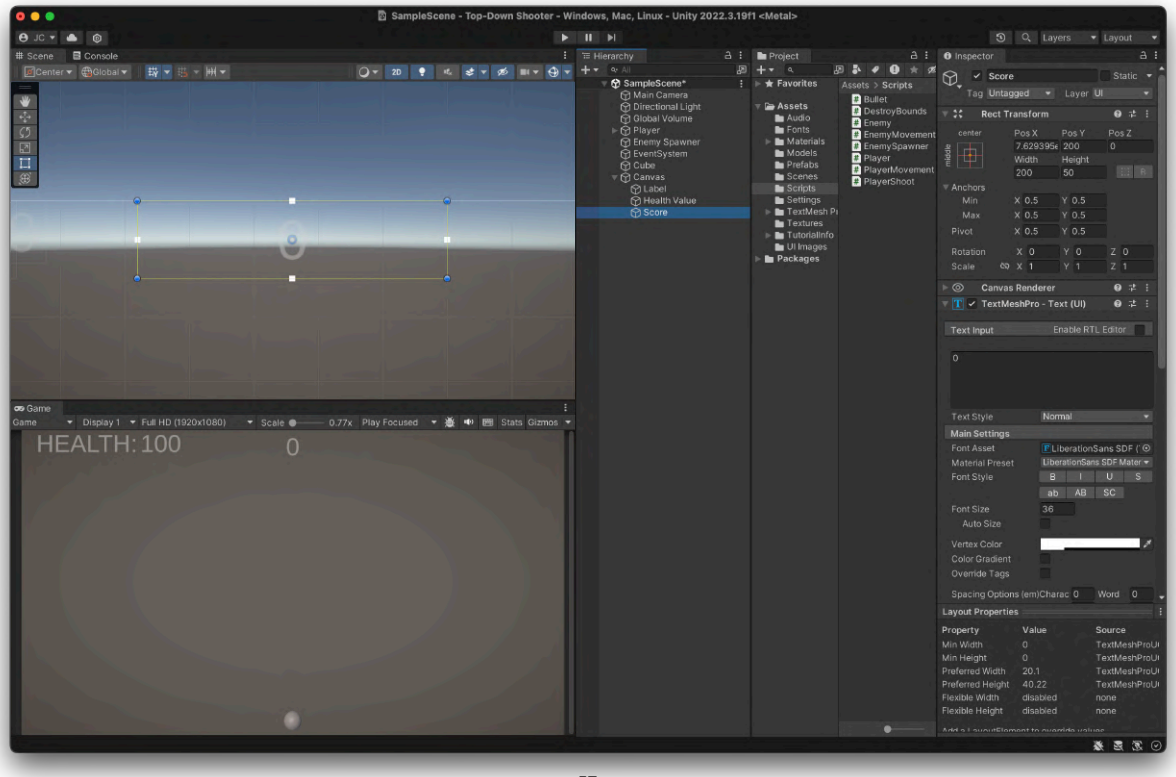
  1. Create a new **Canvas** object and change the **UI Scale Mode** to **Scale With Screen Size** in the **Canvas Scaler** component. Create three **Text-TextMeshPro** UI game objects inside the **Canvas**. Import additional packages if requested by Unity.

  2. Now using **2D** mode and double-click the **Text-TextMeshPro** UI objects. Place the text objects on the top of the screen using the **Rect Tool**. These three text objects are going to show the health label, the current health value, and the score of the player.


SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0
Color




SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0

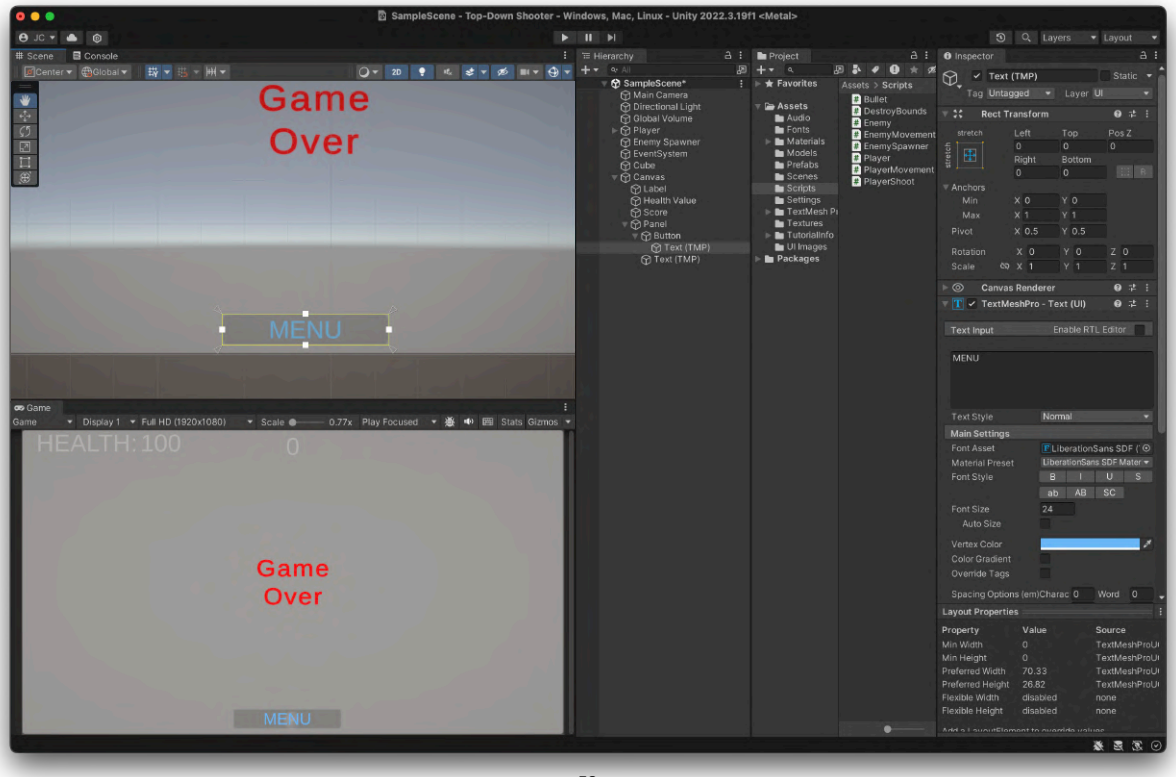




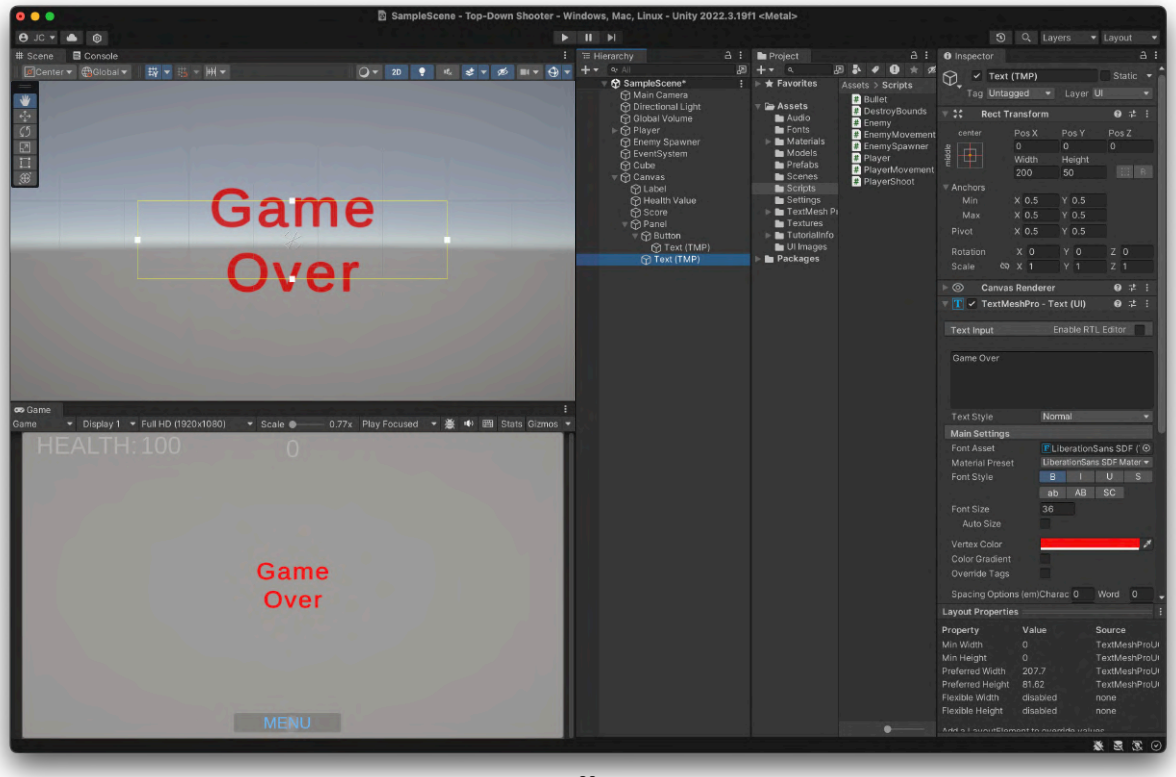

# Game UI

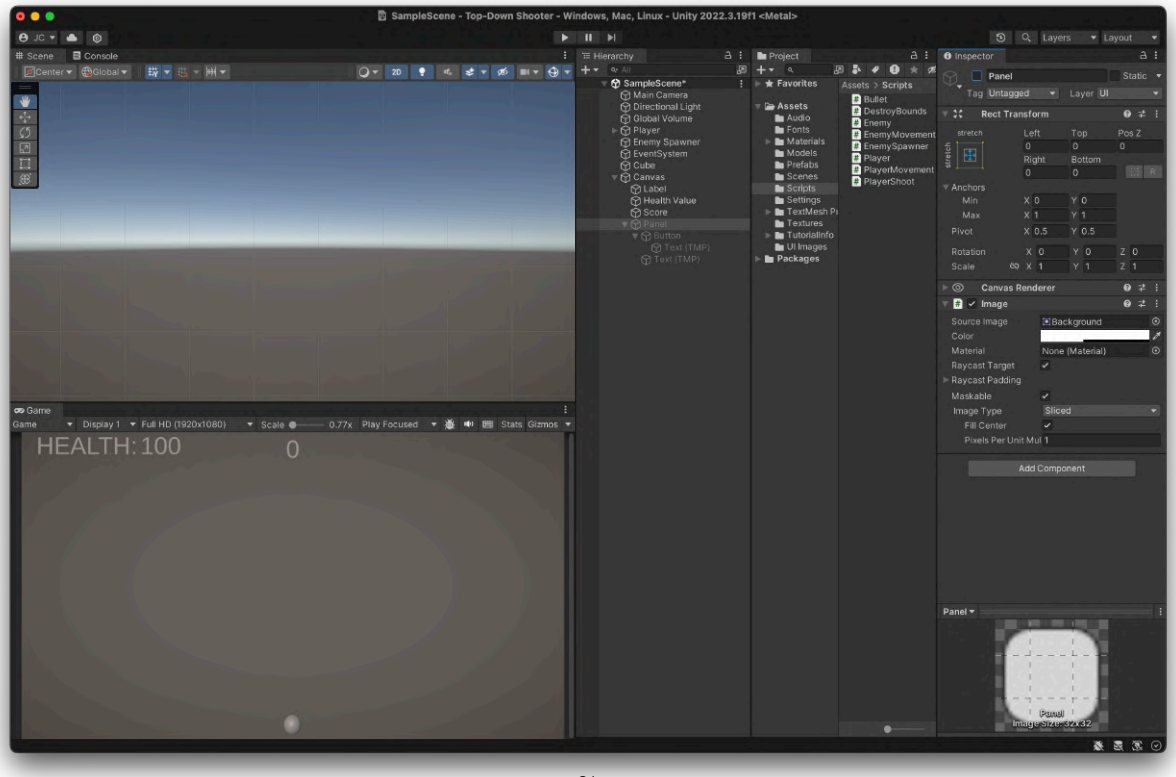
3. Inside the canvas, create a UI **Panel** and add a **Button** and a **Text-TextMeshPro** object into the **Panel.** Change the text inside the **Button** to **MENU** and the text inside the **Text-TextMeshPro** to **Game Over**. And rename the text objects to **Menu** and **GameOver**. You can also change the color of the **Panel** to be semi-transparent black.

4. Now deactivate the **Panel** in the Inspector window.


SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
Game Over
MENU
HEALTH: 100
0
Game Over
MENU



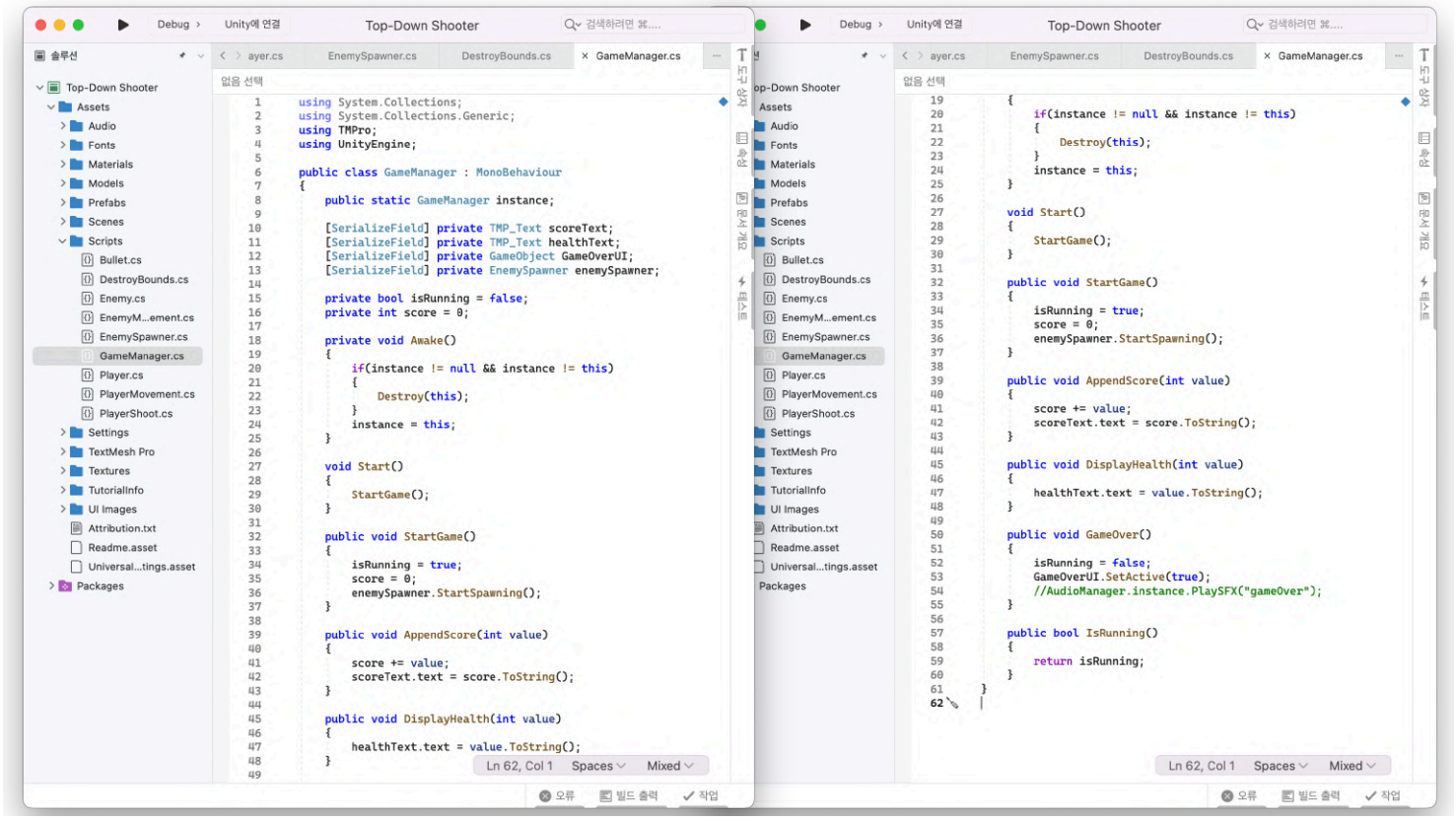

SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
Game Over
HEALTH: 100
0
Game Over
MENU





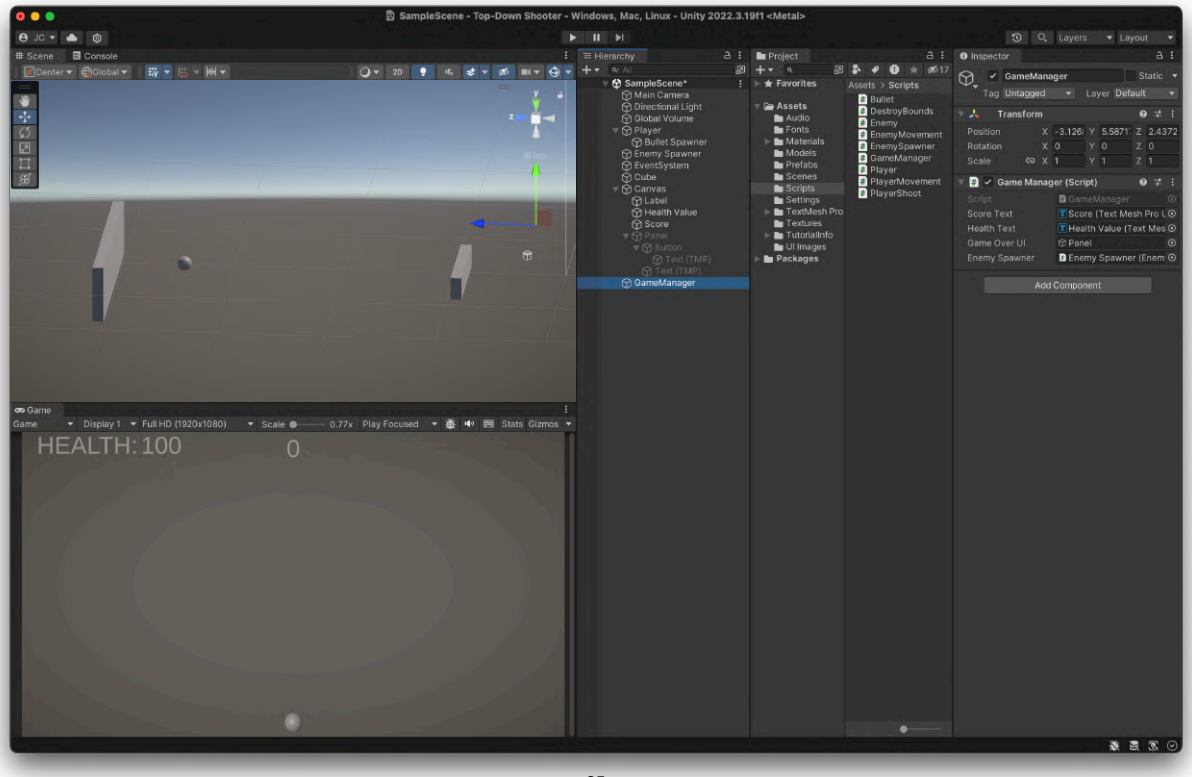

# Game manager

- Game manager should be a component that can be referenced by any other component without assigning it manually.
- And also, there can only be one game manager in the scene.
  - In order to this, we can use a ***Singleton*** design pattern along with public access to its methods.
- Create a new C# script and name it as **GameManager**.
  - The **GameManager** can perform the following tasks: Start the game, Append and display the score, Display the health of the player, and Ends the game.




# Game manager

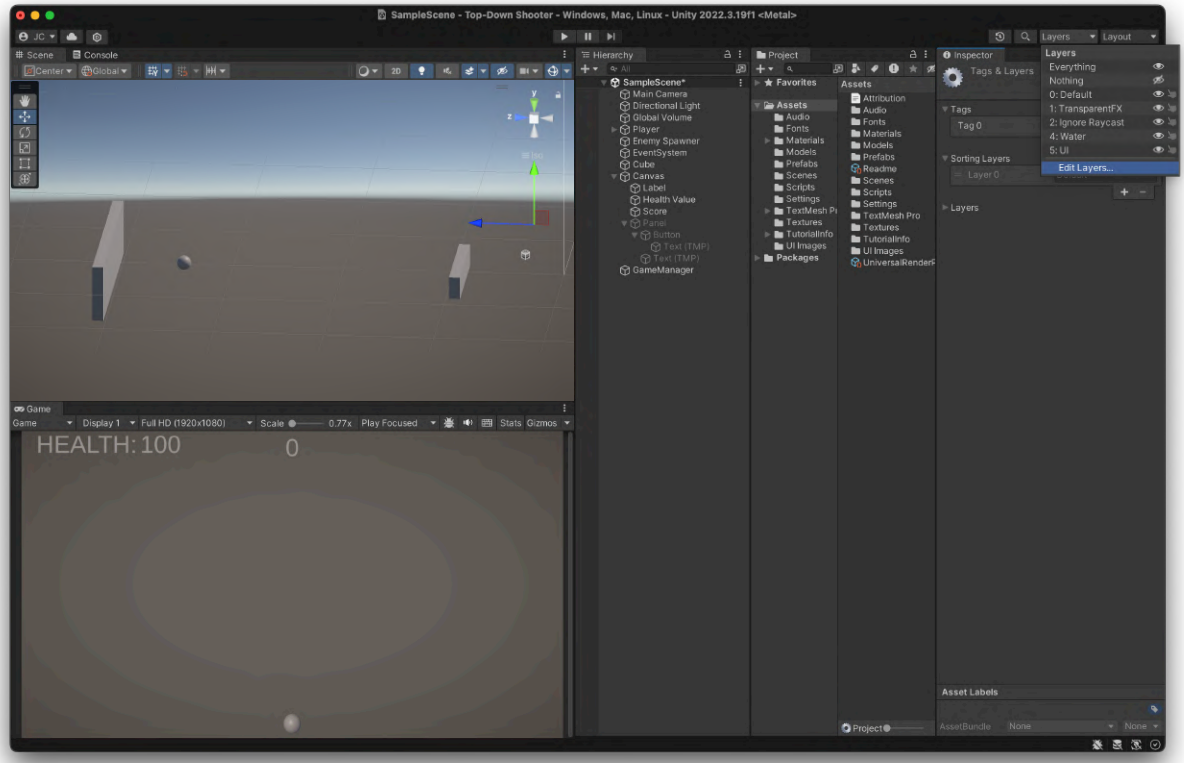
- Attach the script to a new empty game object in the scene and rename the game object as **GameManager**.
- Drag and drop the necessary UI components from the **Hierarchy window** to the **GameManager**.



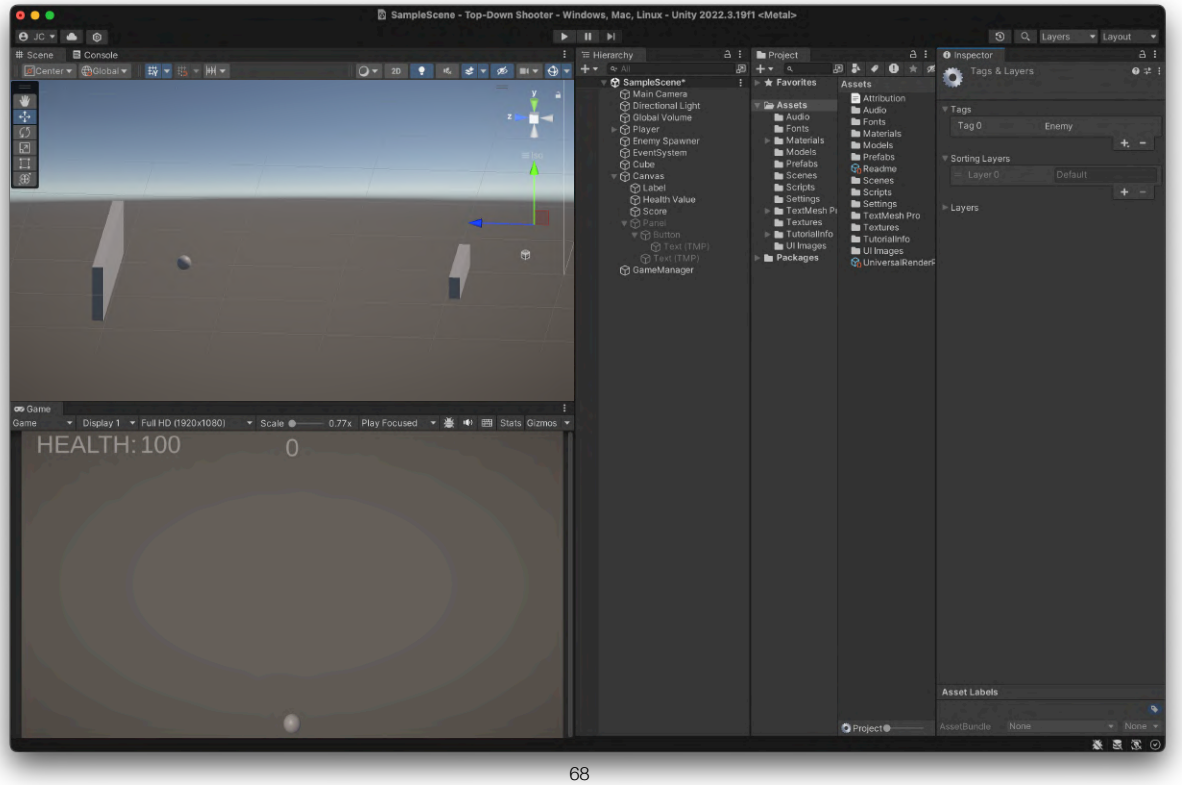



# Game manager

- Now we need to add the proper tags to game objects for the game to work properly.
  - Select the **Layers** dropdown at the top right corner of the Unity Editor, and select **Edit Layers**.
  - Now from the tree view, expand **Tags** and add the **Enemy** tag by clicking the plus (**+**) button.


SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0

67

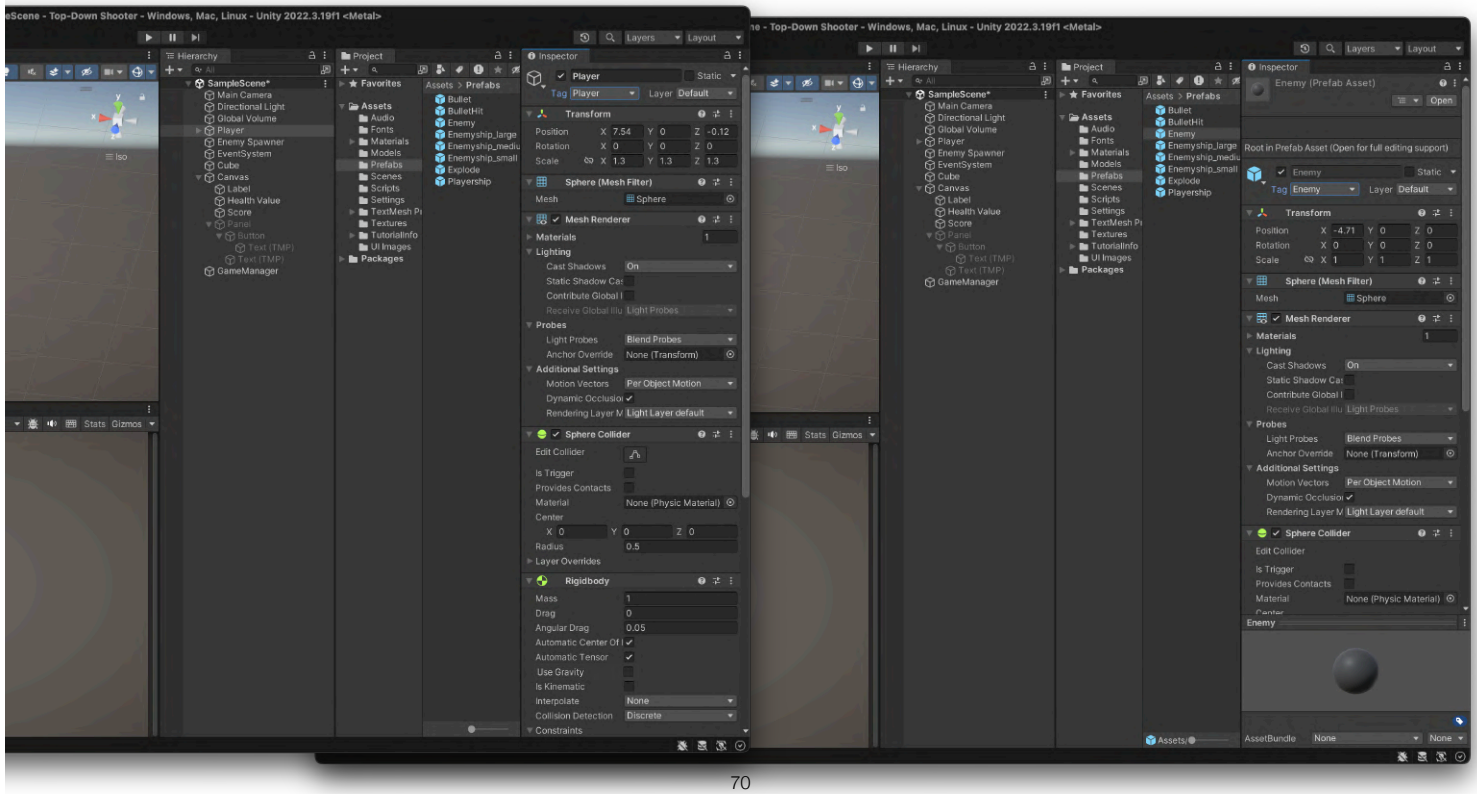

SampleScene - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0

68

# Game manager

- Select the **Player** object and assign the **Player** tag to the **Tag** of the Inspector window.
- Similarly, select the **Enemy** prefab from the Project window and assign the **Enemy** tag to the **Tag**.
- Finally, uncomment all the **GameManager** reference lines wherever we commented them in the previous scripts.

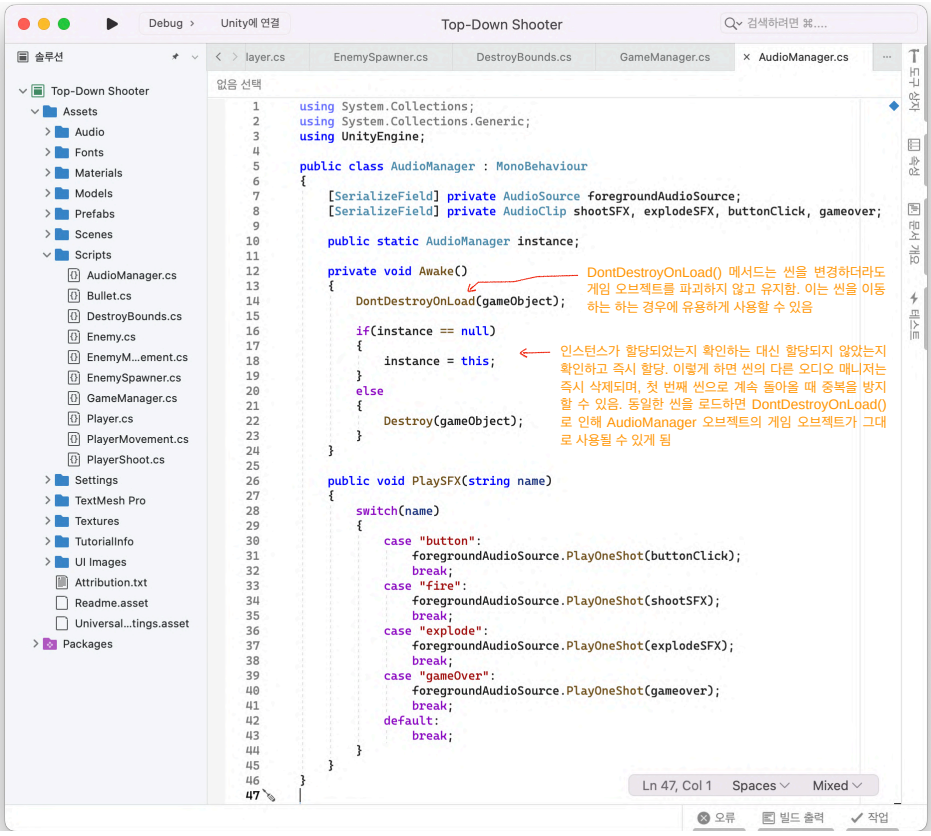




# Adding audio and polishing

- We have been calling a Singleton instance of the **AudioManager** and its **PlaySFX()** method from different other components.
- Create a new C# script and name it as **AudioManager**.
  - We will not be attaching this script to our **Game** scene.
  - Instead, we are going to create a new scene for the main menu and put it in there.

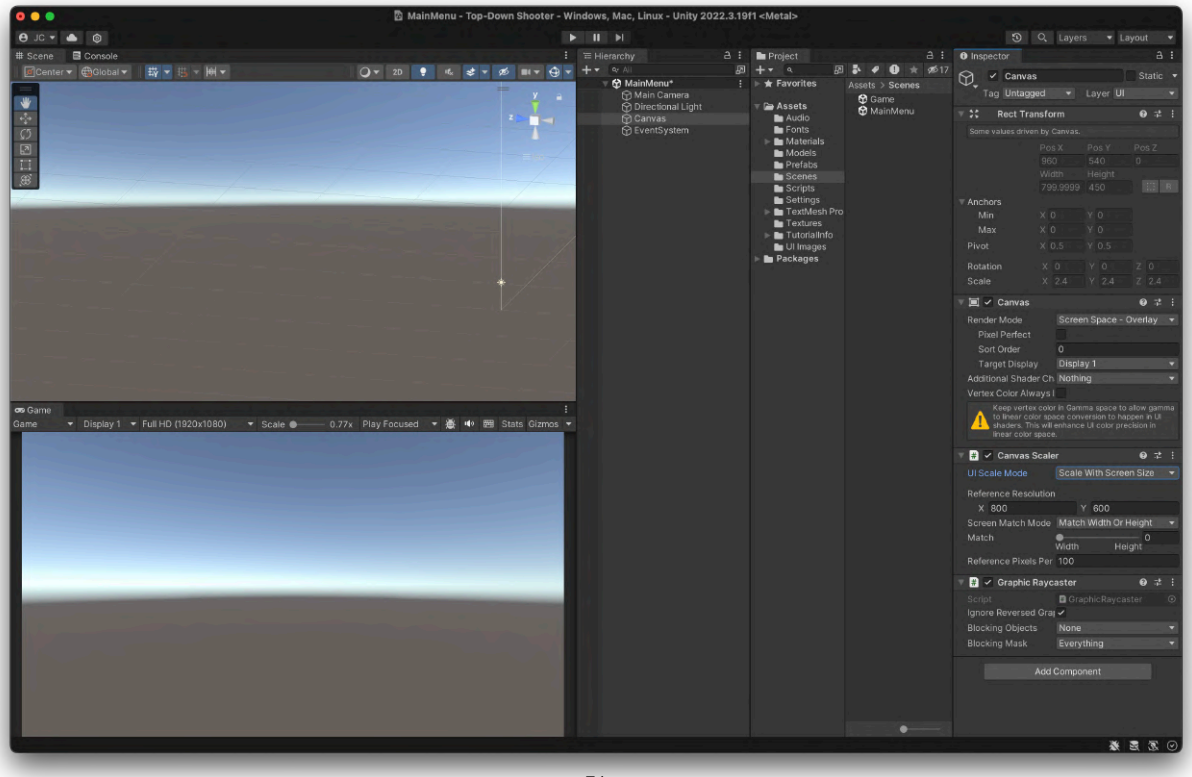


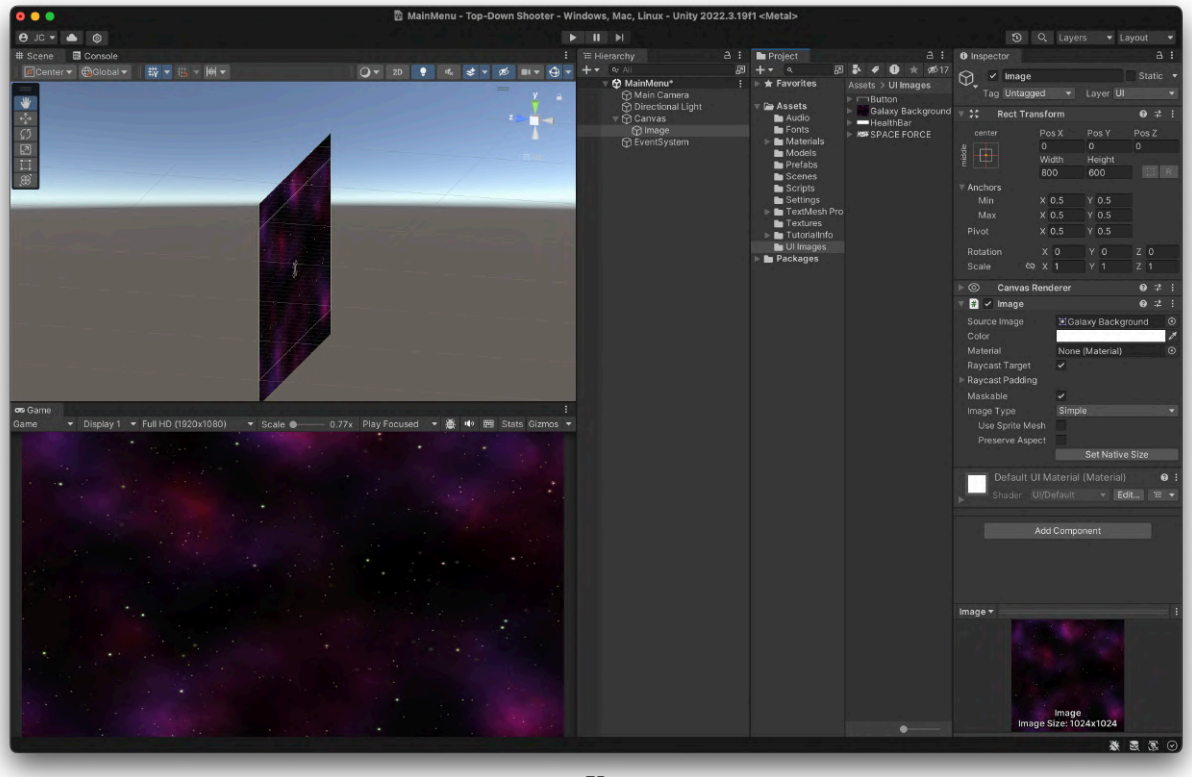


# Adding the main menu

- Rename the **SampleScene** as **Game**. Create a new scene and name it as **MainMenu**.
- Now follow the given steps:
    1. Double-click and open the **MainMenu** scene.
    2. Create a new **Overlay** canvas (Create a new **Canvas** and set the **Render Mode** to **Screen Space - Overlay** in the **Canvas** component) and change the **UI Scale Mode** to **Scale With Screen Size** in the **Canvas Scaler** component.
    3. Inside the canvas, add a UI **Image**. Use the **Rect Transform** to scale the image to cover the whole canvas (**Width: 800**, **Height: 600**). You can use the **Galaxy Background** image from the **Assets** as the background.





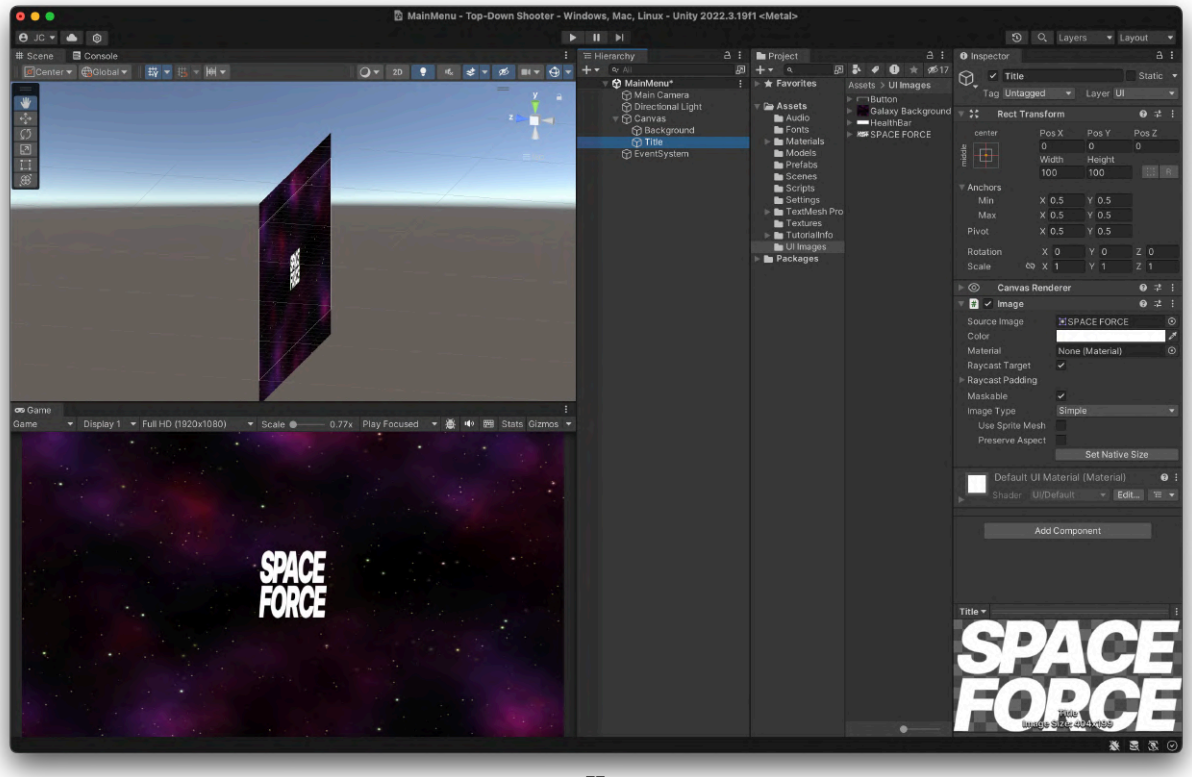




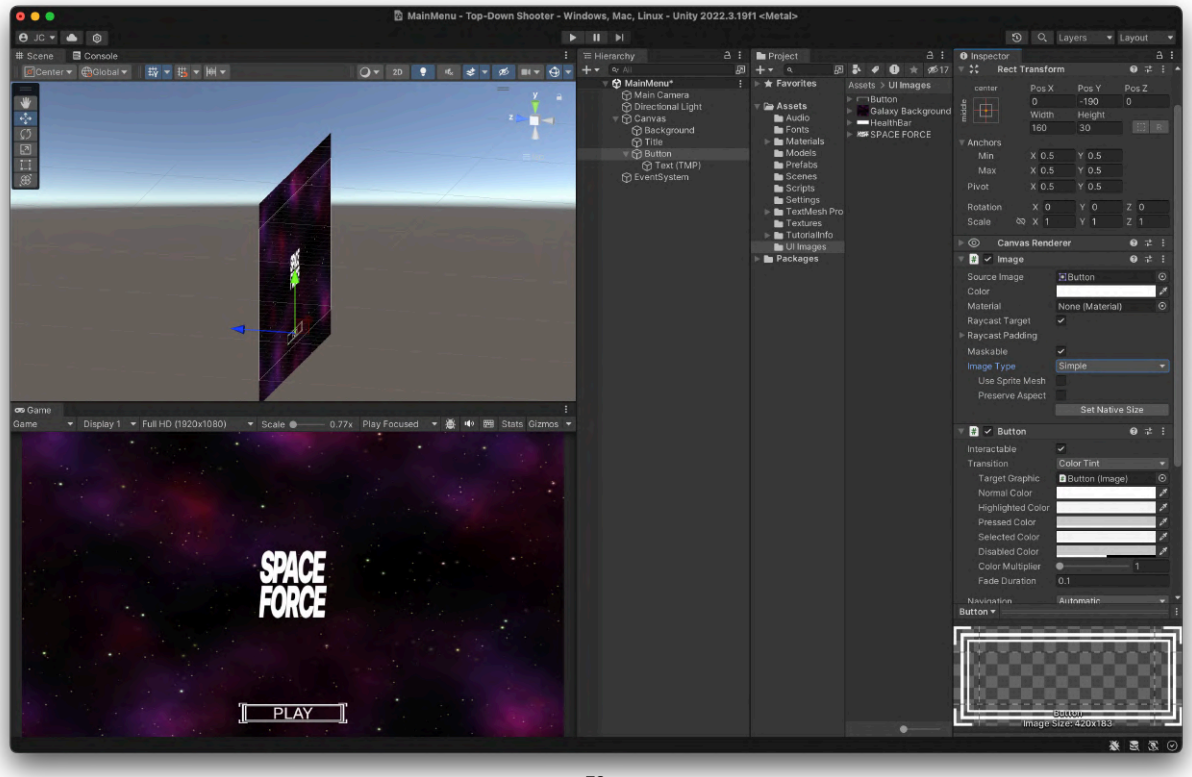

# Adding the main menu

4. Now add a new **Image** object inside the canvas. Use the **SPACE FORCE** image in the **Assets**. You can use the anchoring points to anchor the title image to the center.

5. Add a **Button** inside the canvas and change the text inside the button to **PLAY**. You can also use the **Button** image as the button background from the **UI Images** in **Assets**. Once done, change the **Image Type** in the **Image** component to **Simple**.


MainMenu - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
SPACE FORCE
SPACE FORCE

77

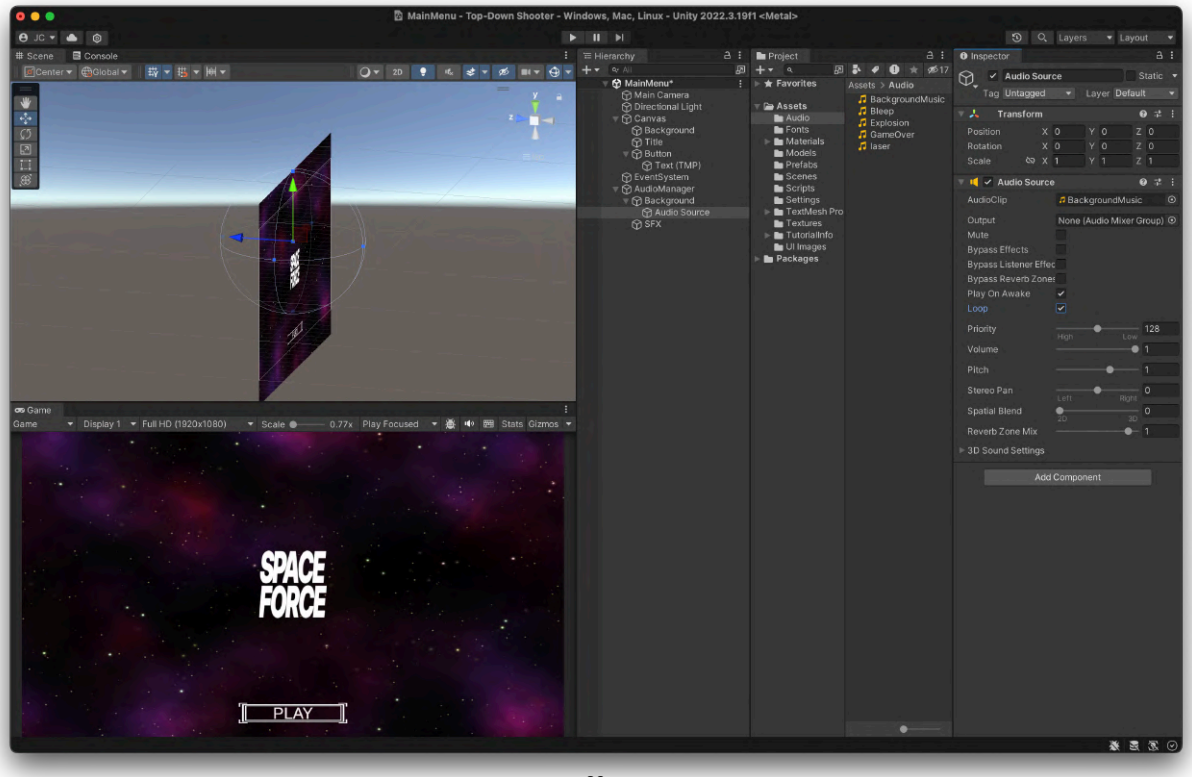

MainMenu - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
SPACE FORCE
PLAY

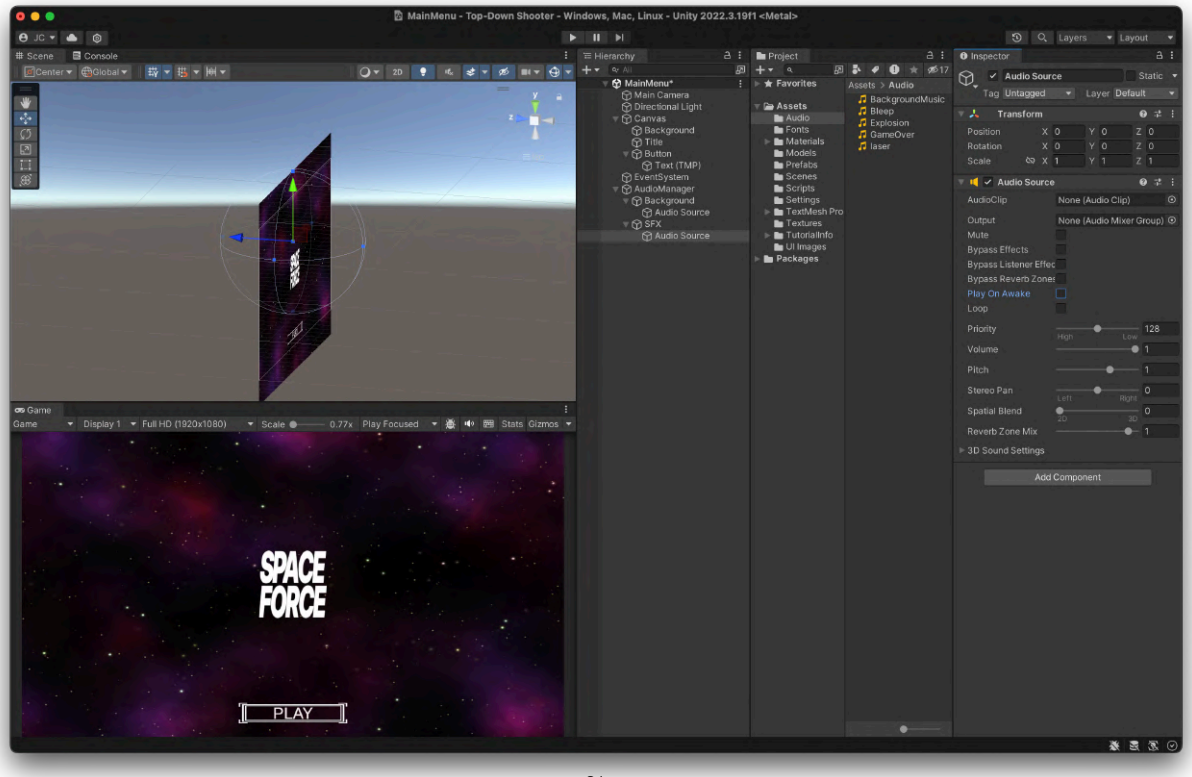
78

# Adding the main menu

- Now let us set up the audio manager. Follow the given steps:
  1. Create an empty game object and name it to **Audio Manager**. Add two empty game objects (**Background** and **SFX**) in the **AudioManager**.
  2. Select the **Background** game object:
     A. Add an **Audio Source** object (**Audio | Audio Source**).
     B. Select the **Audio Source** object and assign the **BackgroundMusic** audio clip from the **Assets** as the **AudioClip** in the **Audio Source** component.
     C. Enable **Play On Awake** and **Loop** in the **Audio Source** component.
     D. Select the **SFX** game object and add an **AudioSource**. Disable **Play On Awake**.





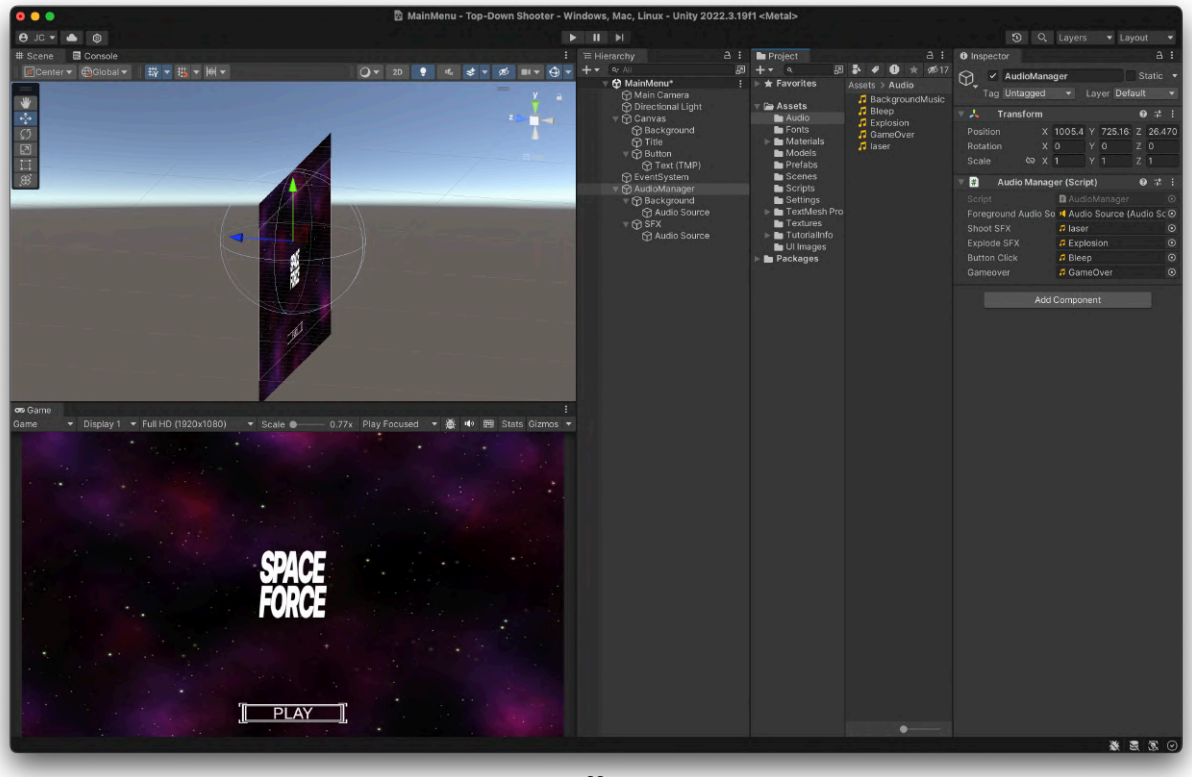





# Adding the main menu

3. Now select the **AudioManager** game object and add the **AudioManager** script component.

4. Assign the **SFX** game object's **Audio Source** object as the **Foreground Audio Source**. You can assign other audio clips from the **Assets**.

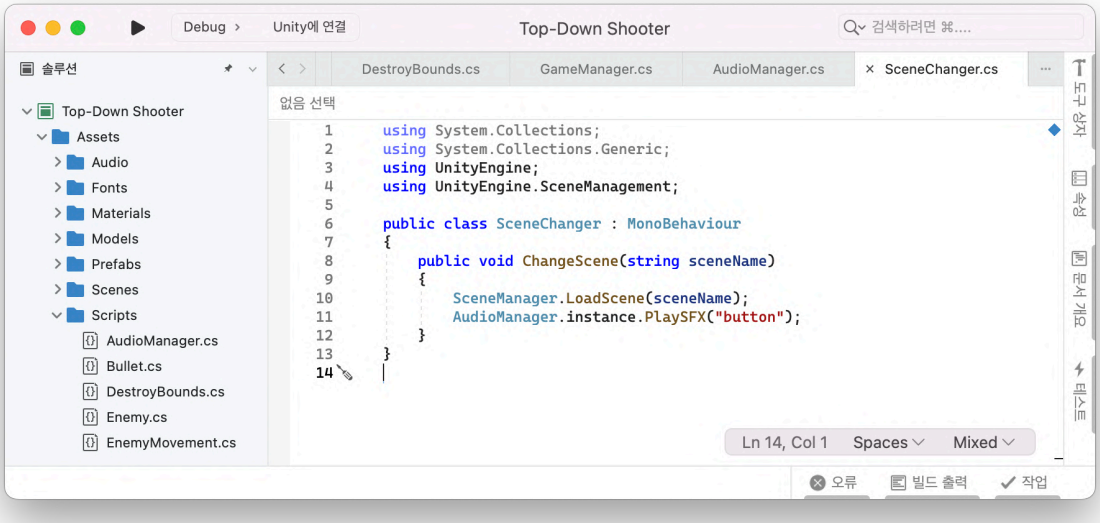





# Switching scenes

- Finally, we need a way to switch from one scene to another.
- This is to be performed by the **PLAY** button in the **MainMenu** scene and the **MENU** button in the **Game** scene.
- Let us create a C# script and name it **SceneChanger**.

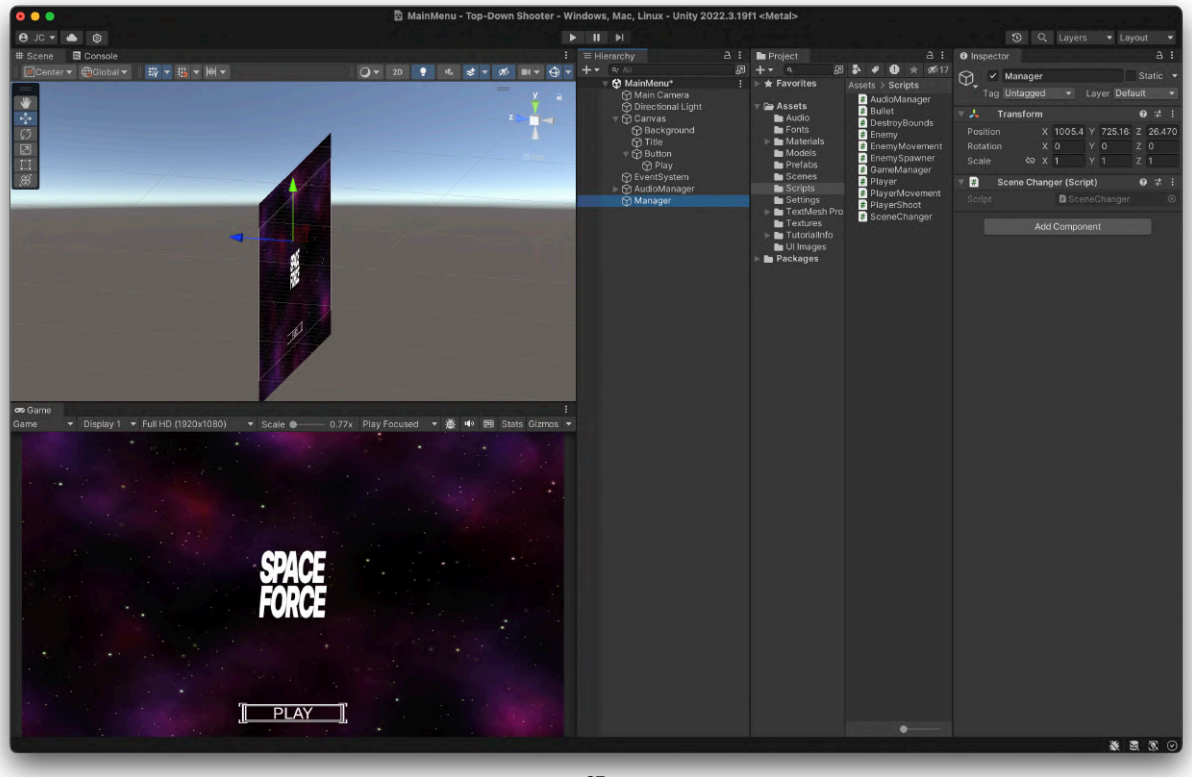
# Switching scenes






# Switching scenes

- Once the script is created, follow the given steps:
  1. We will be using the **SceneManager** to switch the scene by the name of scene. Additionally, we call the **AudioManager** to play the button sound effect.
  2. Go back to Unity, attach the **SceneChanger** script to a **Manager** empty game object.

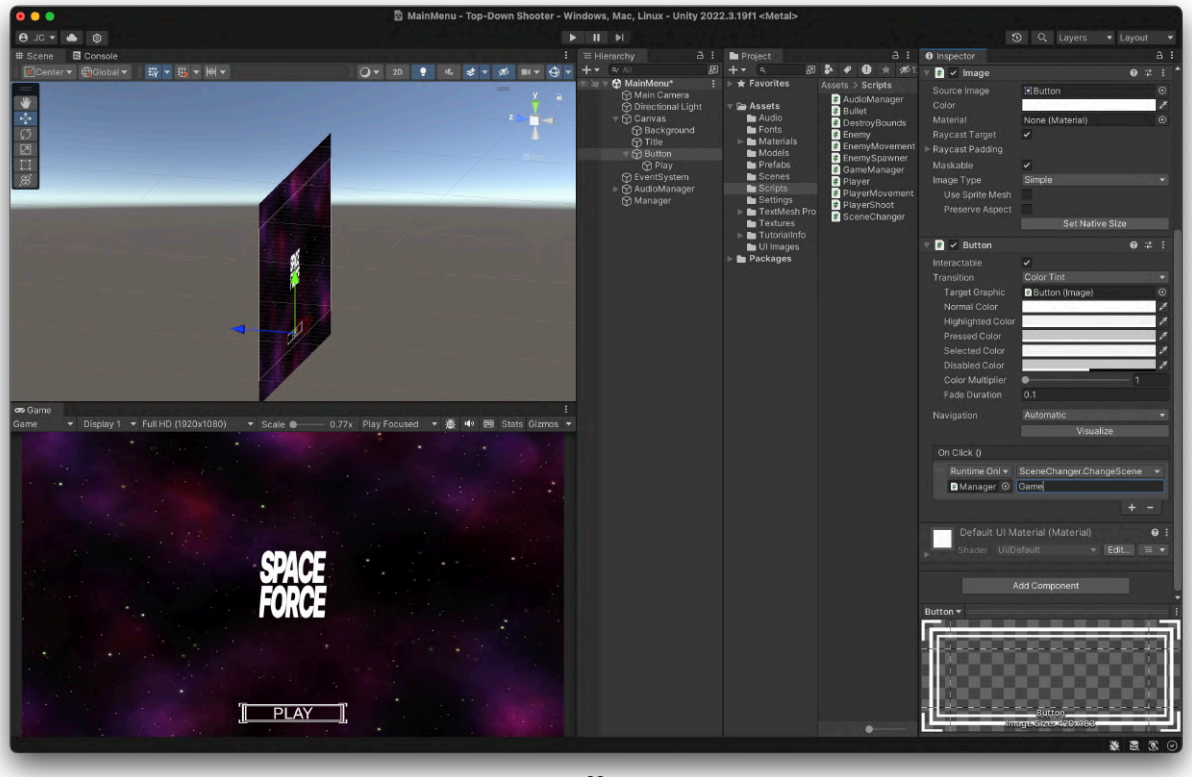





# Switching scenes

3. Select the **Button** object and under **OnClick()** event, click the **+** (plus) button and drag and drop the **Manager** game object on top of **None(Object)** prompt in the **Button** component. Select the **SceneChanger** component from the drop-down list to the right of the event, and then select the **ChangeScene()** method.

4. Now since the parameter requests a string, the UI will show a text box to add the name of the scene. Since we want to switch to the **Game** scene, type **Game** in the text box.



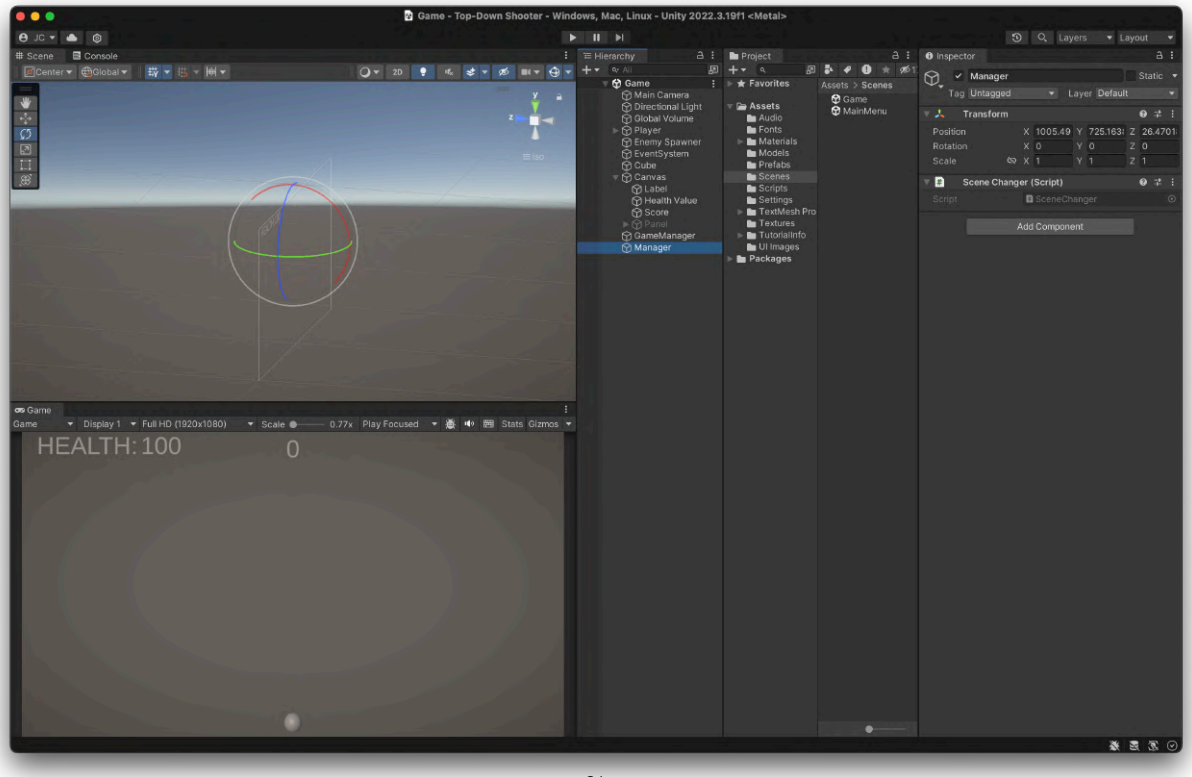



# Switching scenes

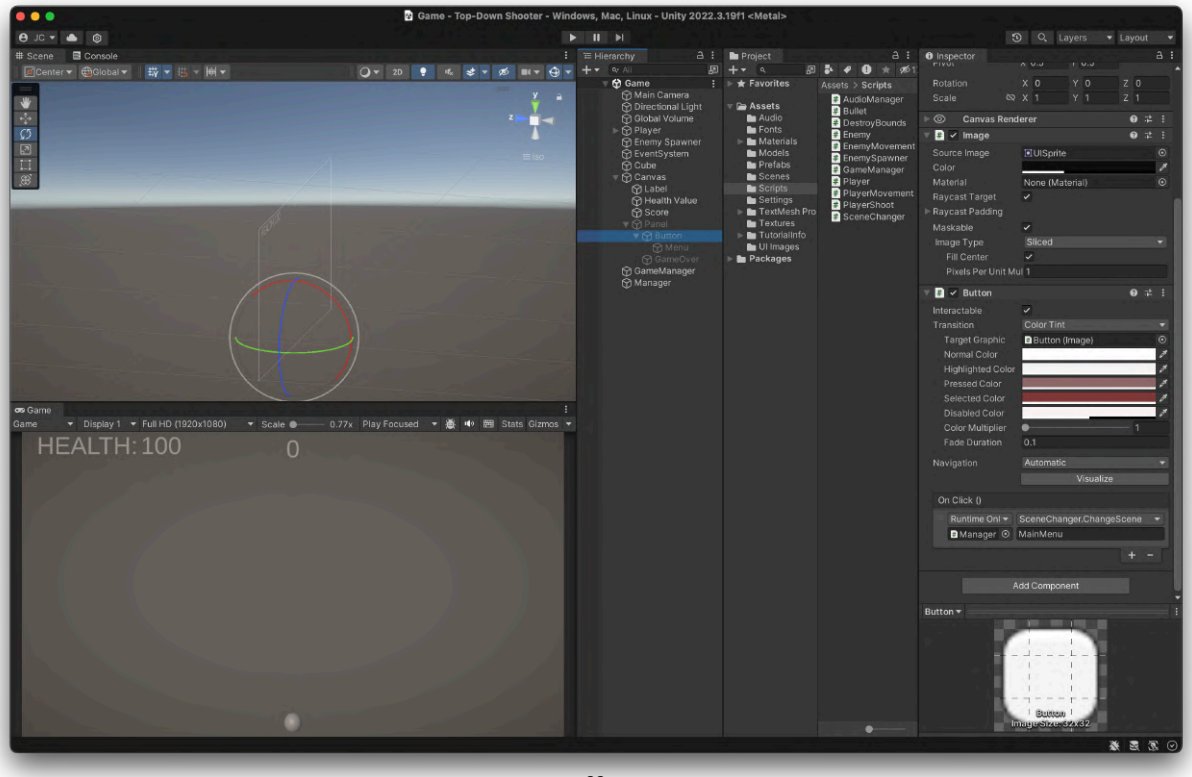
5. Now go to the **Game** scene, create a **Manager** game object and assign it the **SceneChanger** script, just as you did in the **MainMenu** scene, and assign the same with the **Manager** object on the **Menu** button under **Panel** object. However, in this case, we type **MainMenu** in the text input so that pressing the **Menu** button will switch to the **MainMenu** scene.

6. The **SceneManager** script uses the scenes listed in the **Build Settings** in order to switch from one to another. Therefore, we need to add both scenes to the **Build Settings**.

7. Click **File** | **Build Settings**.


Game - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0



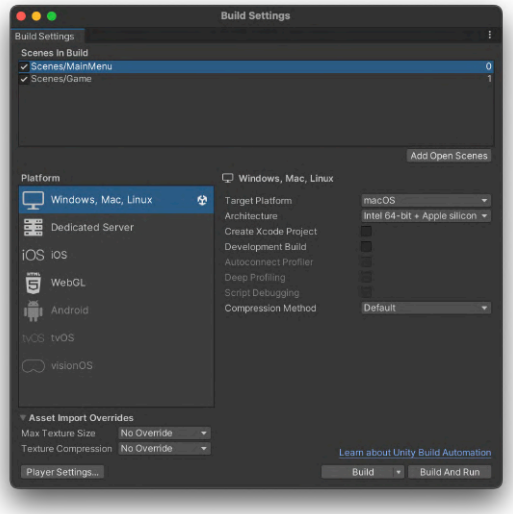

Game - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0

# Switching scenes

8. From the **Build Settings** window, drag and drop the **Game** and the **MainMenu** scene into the **Scene In Build** panel. Make sure to keep the **MainMenu** scene on top of the list.



# Switching scenes



# The next steps

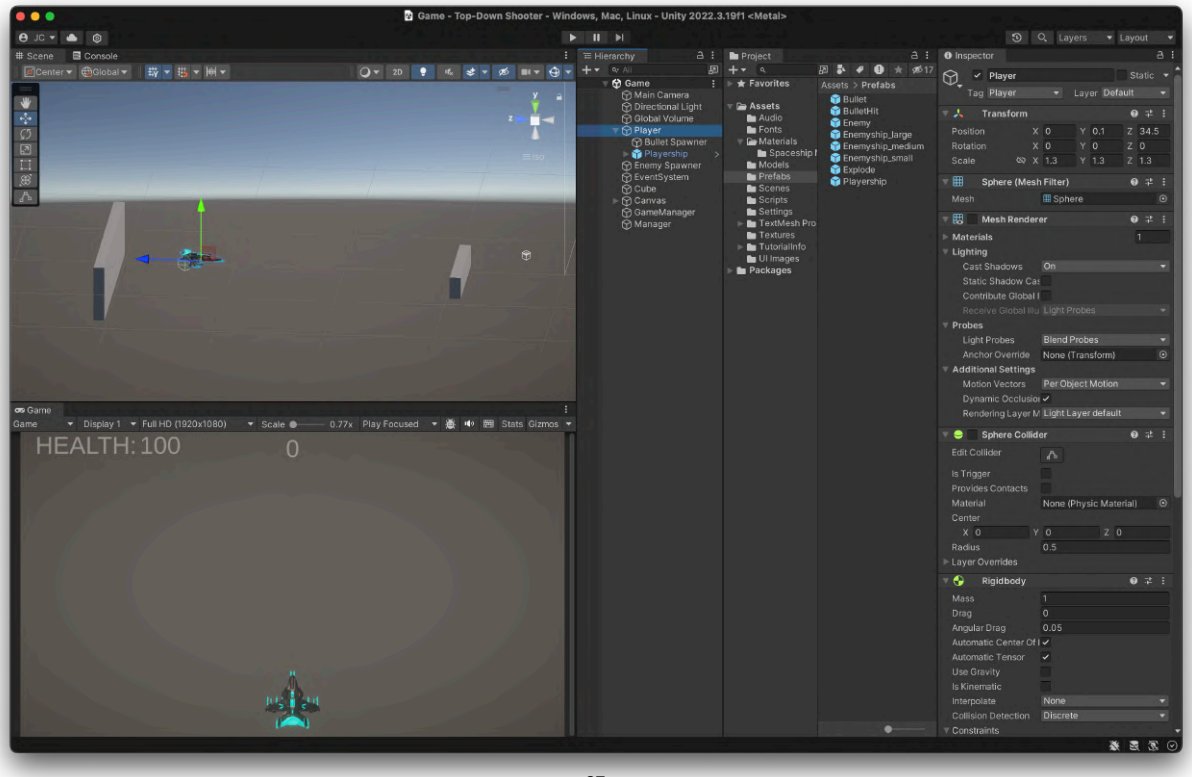
- Now we can click play from the **MainMenu** scene and play the game.
- However, the models do not look nice, and the liveliness of the game is not yet achieved.
- The finished projects has additional features added to the game, such as:
  - Auto-scrolling background in the game view.
  - 3D models for enemies and the player.
  - Adding difficulty by increasing the health of enemies over time.
  - Spawning different enemies with different strengths.



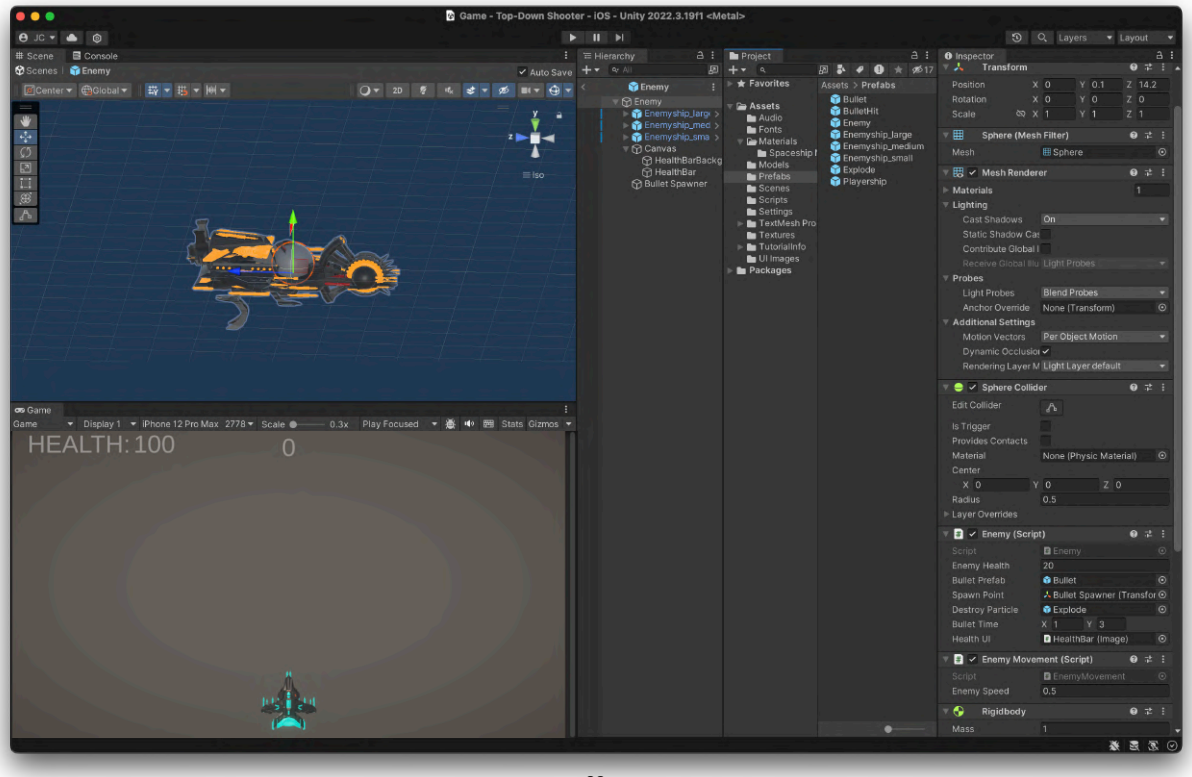



# The next steps

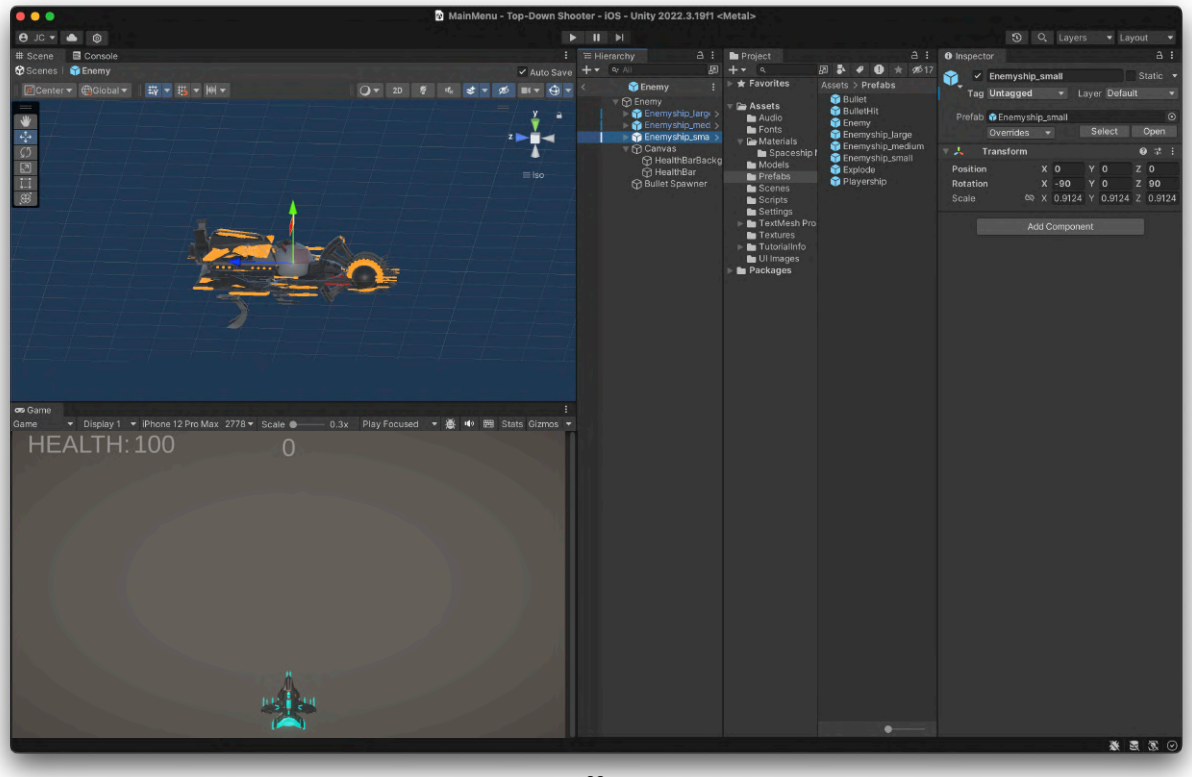
- Let us apply the 3D models from the **Playership** and the three **Enemyship** prefabs to the player and enemies.
  - Select the **Player** game object and from **Prefabs** under **Assets**, drag and drop the **Playership** under the **Player** game object. Also make sure to disable the **Mesh Render** and the **Sphere Collider** of the **Player** game object.
  - Double-click on the **Enemy** prefab and drag and drop the **Enemyship_large**, **Enemyship_medium**, **Enemyship_small** prefabs below the **Emeny** prefab. And make sure the **Position** of all three of these **Enemyship** prefabs should be set to (**0**, **0**, **0**).


Game - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0

97


Game - Top-Down Shooter - iOS - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0

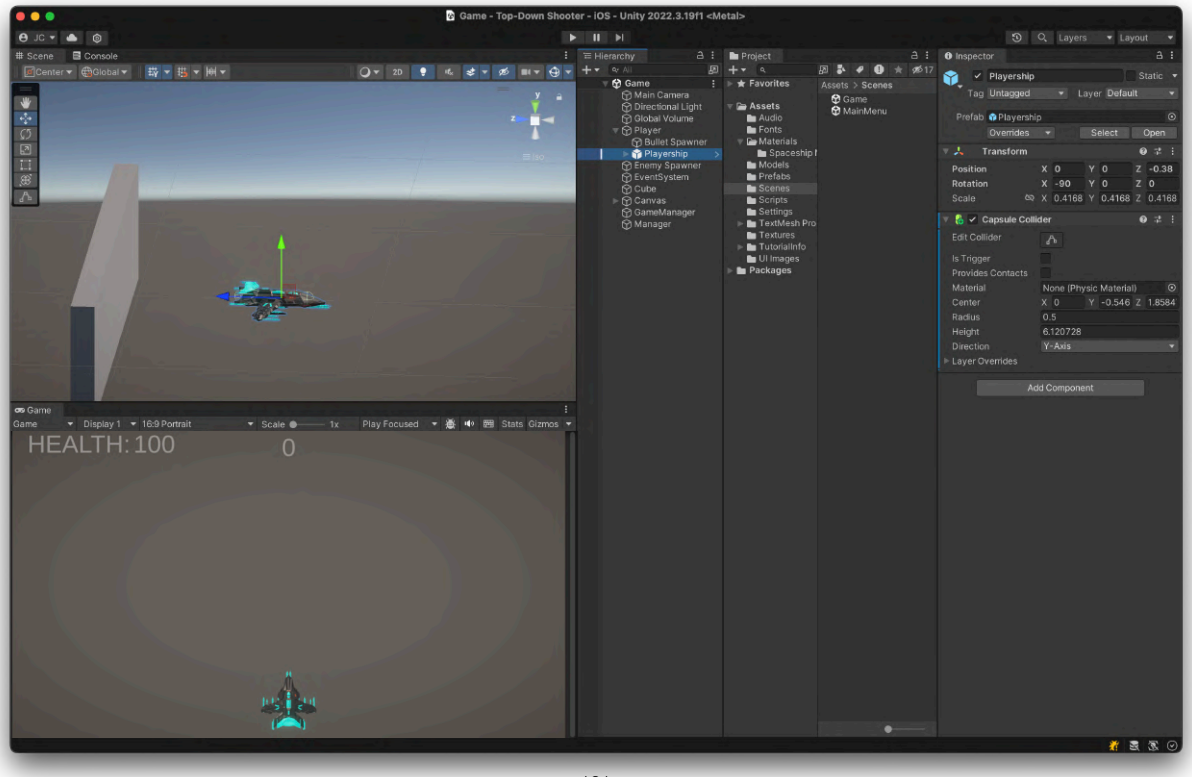
98



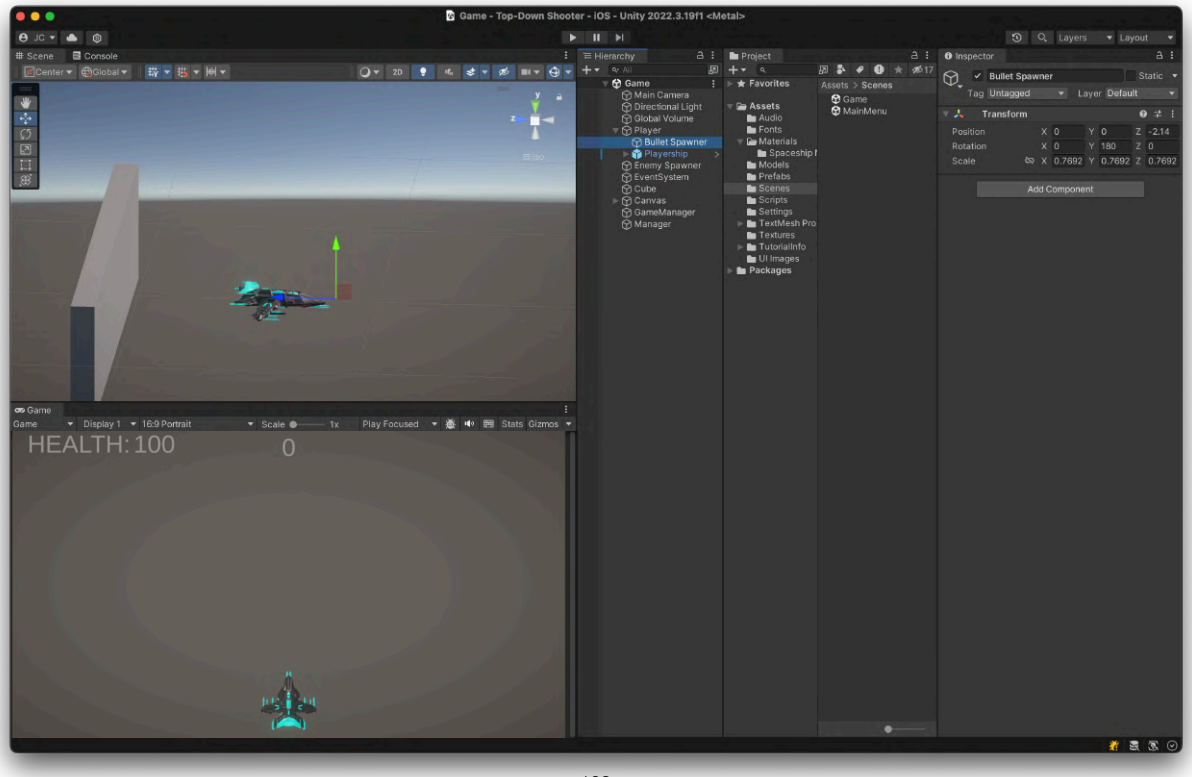



# The next steps

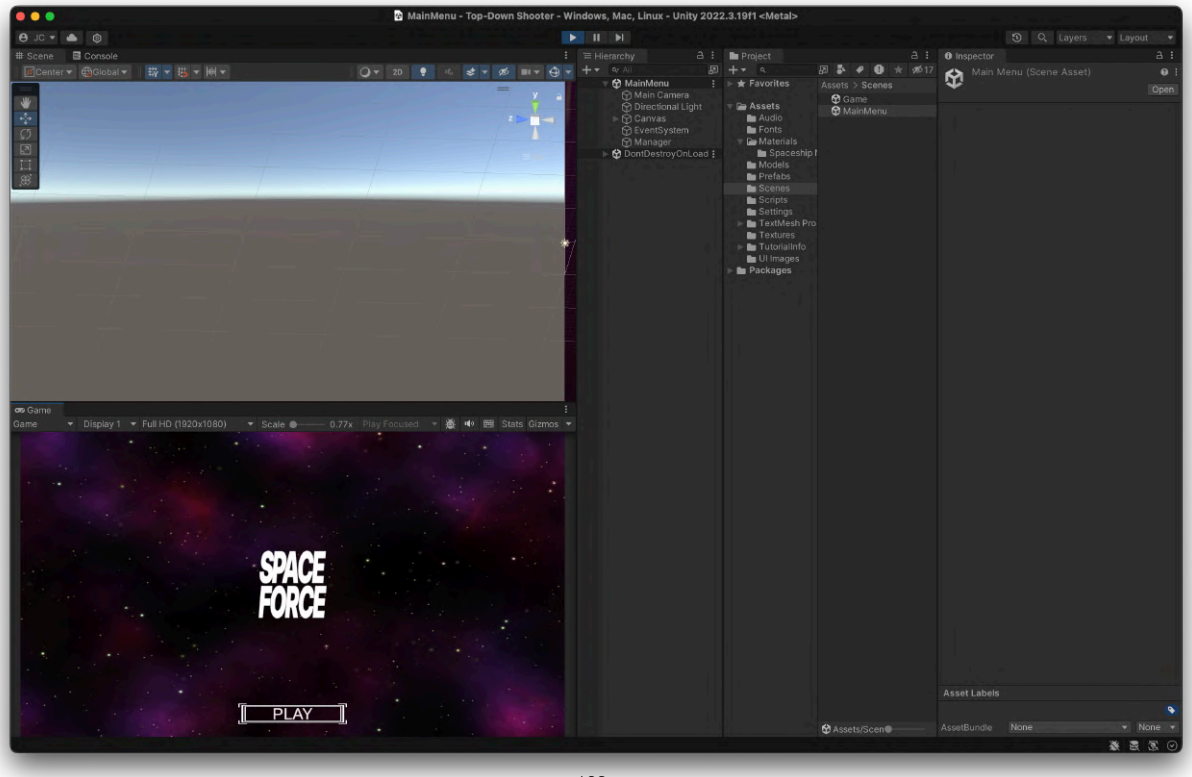
- Now, select the **Playership** under the **Player** game object and add the **Capsule Collider**.
- And Make sure the the **Position** of the **Bullet Spawner** game object is outside of the **Capsule Collider** area of the **Playership**.
- Once you have done everything up to this point, press the **Play** button to play the game.


Game - Top-Down Shooter - iOS - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0
Playership
Capsule Collider
Transform

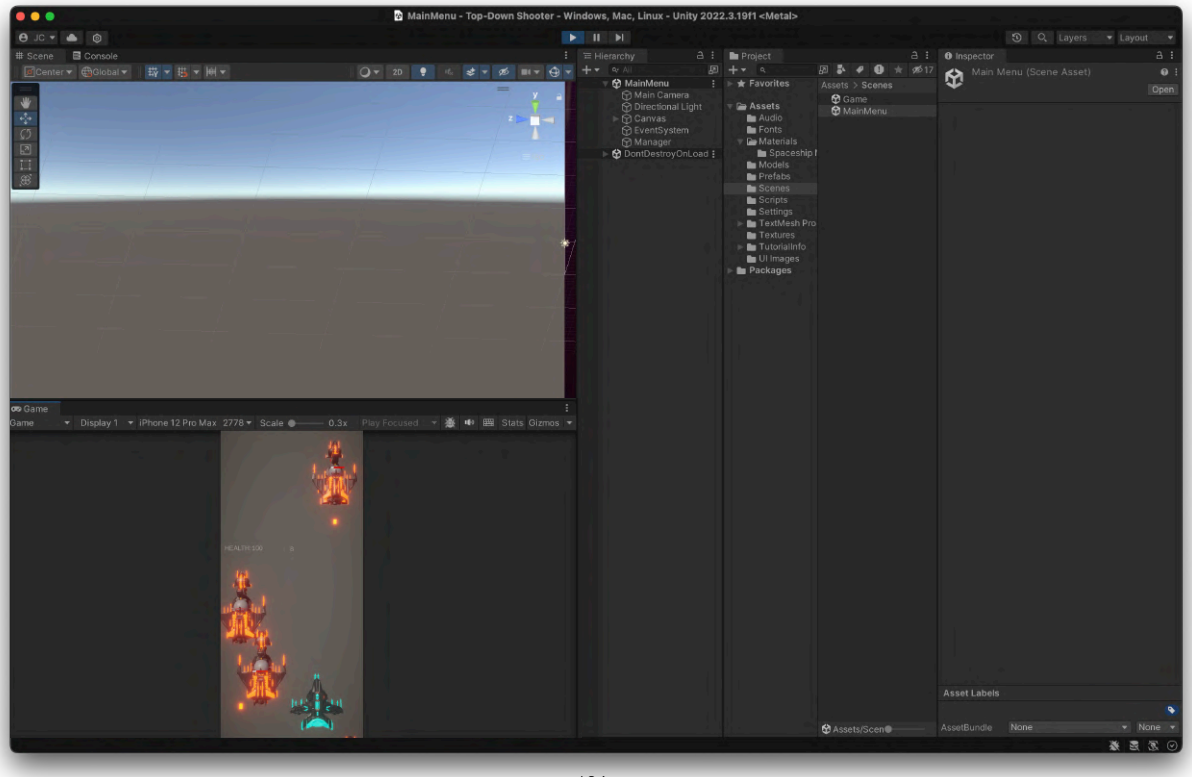
101


Game - Top-Down Shooter - iOS - Unity 2022.3.19f1 <Metal>
HEALTH: 100
0
Bullet Spawner
Transform

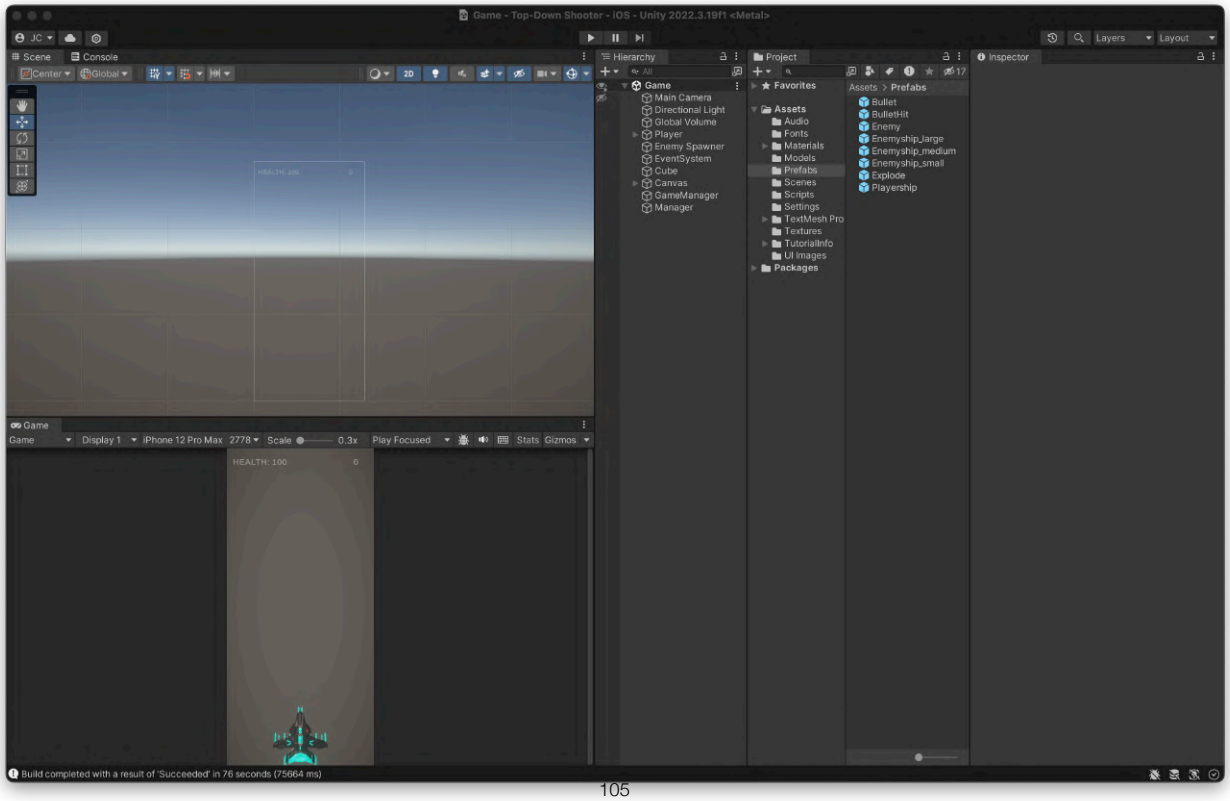
102


MainMenu - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>
SPACE FORCE
PLAY

103


MainMenu - Top-Down Shooter - Windows, Mac, Linux - Unity 2022.3.19f1 <Metal>

104






# The next steps



# Conclusion

- We went through the precess of adding game objects, developing C# scripts that work with each other, design patterns, using prefabs with collisions, simple scoring mechanisms as well as touch input.
- In the next chapter:
  - We will learn to build the game on Android and Apple devices and to play properly with the touch input.
  - From the next chapter onwards, we will focus on AR integrations with Unity.



# Key points

- **Instantiate** method is used to spawn game objects in the scene.
- **OnCollisionEnter** method detect collision if a **Rigidbody** and a **Collider** are present in both objects.
- **OnTriggerEnter** detects triggers that do not have a force impact.
- Any game object can be converted to a prefab and can be instantiated from the project window during run time.
- Input-based methods can be used in update methods to capture mouse and touch inputs.

# Key points

- **IEumerators** can be used to run coroutines within Unity which works similarly to parallel processes alongside the main application.
- Game UI can be overlay or world space. Overlay UI's render in screen space, while the world space UI uses the 3D scene to render the UI elements.
- The audio source can play on audio clip at a time.
- Buttons in unity use **UnityEvents**, such as **OnClick**, to capture events and execute methods directly from the Inspector window.
- Unity UI images can use filled, tiles, stretched, and sliced methods to present the final texture.



